Trabajo Práctico 1:

Problema de la mochila



Teoría de Algoritmos - Curso Echevarría

75.29/95.06/TB024

Integrantes:

- Finazzi Franco (113513)
- Florio Miceli Augusto Martin (113528)
- Larocca Gigirey Lucas Adrian (114218)
- Superno Lautaro (114524)

Índice:

Índice:....2
Introducción:....4
1. Fuerza bruta vs Backtracking....6
1.1 Supuestos....6
1.2 a. Pseudocódigo Fuerza Bruta y Backtracking....7
b. Estructuras de datos utilizadas....11
1.3 Seguimientos....13
1.4 Complejidad....15
1.5 Set de datos....17
1.6 Tiempos de ejecución....18
1.7 Informe de Resultados....22
2. Greedy....24
2.1 Supuestos....24
2.2 a. Pseudocódigo Greedy....25
b. Estructuras de datos utilizadas....26
2.3 Seguimientos....26
2.4 Complejidad....29
2.5 Set de datos....30
2.6 Tiempo de ejecución....30
2.7 Informe de Resultados....31
3. Programación dinámica....32
3.1 Supuestos....32
3.2 Diseño....33
3.2.1 Planteo tradicional: maximizar el beneficio para un peso fijo W:....34
3.2.2 Planteo alternativo: minimizar el peso con un beneficio fijo B....36
3.2.3 Seguimientos....39
3.3 Complejidad....42
3.4 Sets de Datos....44
3.5 Tiempos de ejecución....45
3.6 Informe de resultados....47
3.7 Conclusiones....49
4. Programación lineal....50
4.1 Supuestos....50
4.2 a. Ecuación fundamental y pseudocódigo....50
b. Estructuras de datos utilizadas....51
4.3 Seguimiento....52
4.4 Complejidad....54
4.5 Set de datos....55
4.6 Tiempos de ejecución:....55
4.7 Informe de los resultados....56

Conclusiones.....57
Referencias.....60

Introducción:

El problema de la mochila consiste en que, dada una mochila con una capacidad W y una cantidad de N elementos, cada uno con un peso Wi y un valor Vi, se debe determinar qué combinación de objetos guardar de forma tal que el valor total de los objetos incluidos sea máximo sin superar la capacidad límite.

En este informe se abordará este problema implementando diferentes estrategias, incluyendo fuerza bruta, backtracking, greedy, programación dinámica y programación lineal. Cada implementación contará con un análisis que profundizará en sus características y complejidad, además de proporcionar los resultados prácticos de ejecución de cada una. Al final, se presentará una comparación general entre todos los enfoques evaluando sus rendimientos y las soluciones obtenidas para cada uno.

Todos los algoritmos se implementaron en Python 3.9.6, por lo que no requieren compilación (Las mediciones del Ejercicio 3 se realizaron con Python 3.14.2.). Las bibliotecas utilizadas y sus versiones se listan en el archivo TP1/requirements.txt y se instalan, desde la carpeta TP1/, con:

pip install -r requirements.txt

Cada ejercicio se encuentra en su propia carpeta dentro de TP1/ y sus comandos se ejecutan desde ella:

- Ejercicio 1 (Ej1_FB_BT): python3 mochila_fuerza_bruta.py sets/mochila20.txt y python3 mochila_backtracking.py sets/mochila20.txt resuelven un set individual; python3 medir.py mide los tiempos de todos los sets y guarda los resultados en resultados/; python3 graficos.py genera los gráficos en graficos/.
- Ejercicio 2 (Ej2_Greedy): python3 mochila_greedy.py sets/mochila100.txt resuelve una instancia individual; python3 medir.py ejecuta las mediciones sobre todo el rango de datos ($N = 100$ a 500.000) y guarda los tiempos en resultados/; python3 graficos.py genera el gráfico semilogarítmico de tiempos medidos vs. cota teórica $O(N \log N)$ en graficos/.
- Ejercicio 3 (Ej3_Programacion_Dinamica): python3 mochila_programacion_dinamica1.py sets/mochila20.txt resuelve un set con el planteo tradicional, y python3 mochila_programacion_dinamica2.py sets/mochila20.txt [B] lo resuelve con el planteo alternativo (si no se indica B, se usa

el beneficio óptimo del tradicional, si no se indica el set, se usa sets/mochila120.txt). Aclaración: En el segundo algoritmo, para indicar B también hay que indicar el set, por ejemplo: “python3 mochila_programacion_dinamica2.py sets/mochila120.txt 3000” ya que el primer argumento debe ser siempre el archivo de set.
python3 graficos_pd.py mide los tiempos de todos los sets y guarda los resultados en resultados/ y los gráficos en graficos/.
python3 generar_sets.py regenera los sets con semilla fija.

- Ejercicio 4 (Ej4_ProgramacionLineal): python3 mochila_programacion_lineal.py para ejecutar con un set aleatorio de 10000 items, python3 grafico_sets.py genera un grafico comparativo con los sets de datos utilizados en el informe

Estas mismas instrucciones se encuentran en el archivo TP1/README.md incluido en la entrega.

1. Fuerza bruta vs Backtracking

1.1 Supuestos

Ambos algoritmos resuelven el problema de la mochila 0/1 y comparten los siguientes supuestos sobre la entrada:

- Capacidad entera positiva ($W > 0$). Con capacidad nula la única solución factible es la mochila vacía y el problema carece de interés.
- Pesos enteros positivos ($p_i > 0$). Además de reflejar el modelo físico del problema, garantiza que el cociente beneficio/peso utilizado por el backtracking esté siempre definido.
- Beneficios enteros positivos ($b_i > 0$).
- Problema de tipo 0/1: cada ítem se incluye completo o no se incluye, como máximo una vez; no se admiten fracciones.
- Ítems que exceden la capacidad: se admiten en la entrada y simplemente nunca forman parte de una solución factible. Este caso se presenta efectivamente en los sets utilizados, ya que crear_mochila.py genera pesos de hasta 200 unidades, por lo que en instancias pequeñas un ítem puede superar por sí solo la capacidad de la mochila.
- Formato de entrada: el generado por crear_mochila.py (primera línea: capacidad; líneas siguientes: pares peso,beneficio).
- Unicidad de la respuesta: si existen varios subconjuntos con el beneficio óptimo, ambos algoritmos devuelven uno solo de ellos. El beneficio óptimo es siempre el mismo, pero el subconjunto elegido podría diferir entre ambos algoritmos. Por este motivo, la verificación de que fuerza bruta y backtracking llegan al mismo resultado se realiza comparando el beneficio óptimo obtenido.

Supuestos y limitaciones propios de cada algoritmo:

- Fuerza bruta: al enumerar los 2^n subconjuntos, su uso es viable solo para instancias pequeñas. En el entorno de medición, $n = 22$ ya requiere más de 6 segundos, y cada ítem adicional duplica el tiempo.
- Backtracking:
    - La poda por cota superior utiliza la relajación fraccional del problema, que solo constituye una cota válida si los ítems se recorren en orden decreciente

de beneficio/peso. Por ese motivo, el algoritmo ordena los ítems al inicio y luego traduce la solución a los índices originales.

- Al ser recursivo, la profundidad máxima de llamadas es n + 1. El límite de recursión se fija en 10 000, suficiente para todas las instancias evaluadas.
- Su eficiencia depende de la instancia: la efectividad de las podas varía según los datos, y en el peor caso el algoritmo explora tantos nodos como la fuerza bruta (ver sección 1.4).

1.2 a. Pseudocódigo Fuerza Bruta y Backtracking

Fuerza bruta

Idea. La fuerza bruta resuelve el problema evaluando todas las soluciones posibles. Como cada uno de los n ítems puede incluirse o no en la mochila, existen 2^n subconjuntos candidatos. Para recorrerlos de forma sistemática se representa cada subconjunto mediante un número entero entre 0 y $2^n - 1$, interpretado en binario como una *máscara*: el bit i vale 1 si el ítem i forma parte del subconjunto y 0 en caso contrario. Por ejemplo, con 3 ítems, la máscara 5 (101 en binario) representa el subconjunto formado por los ítems 0 y 2.

Para cada máscara, el algoritmo calcula el peso y el beneficio total del subconjunto que representa. Si el peso no supera la capacidad y el beneficio supera al mejor encontrado hasta el momento, se registra como nueva mejor solución. Al finalizar el recorrido se han evaluado todas las soluciones posibles, por lo que la mejor registrada es necesariamente la óptima. El algoritmo no descarta ningún candidato de antemano: primero construye cada solución completa y recién después verifica si es factible.

```
FUERZA_BRUTA(capacidad, pesos, beneficios):
    n ← longitud(pesos)
    mejor_beneficio ← 0
    mejor_mascara ← 0                    // mochila vacía: siempre factible

    para mascara desde 0 hasta 2^n − 1:
        peso_total ← 0
        beneficio_total ← 0
        para i desde 0 hasta n − 1:
            si el bit i de mascara está activo:
                peso_total ← peso_total + pesos[i]
                beneficio_total ← beneficio_total + beneficios[i]
        si peso_total ≤ capacidad y beneficio_total > mejor_beneficio:
            mejor_beneficio ← beneficio_total
            mejor_mascara ← mascara

    items ← índices i tales que el bit i de mejor_mascara está activo
    retornar mejor_beneficio, items
```

Backtracking

Idea. El backtracking explora el mismo espacio de soluciones, pero construyéndolas de forma incremental en lugar de generarlas completas. El espacio se organiza como un árbol binario de decisión: en el nivel i se decide si el ítem i se incluye o no en la mochila, de modo que cada nodo representa una solución parcial y cada hoja (nivel n) una solución completa. El algoritmo recorre este árbol en profundidad mediante recursión: ante cada ítem prueba primero incluirlo y luego excluirlo, y al regresar de una llamada recursiva deshace la última decisión para explorar la alternativa. Ese retroceso es lo que da nombre a la técnica (Curso Echevarría, Teoría de Algoritmos, 2026a).

La ventaja respecto de la fuerza bruta es que, en cada nodo, el algoritmo puede determinar que una rama completa no conduce a una solución mejor y descartarla sin recorrerla. Se aplican dos podas:

1. Poda por capacidad. Solo se explora la rama “incluir el ítem i” si el peso acumulado más el peso del ítem no supera la capacidad. Así nunca se construyen soluciones parciales infactibles.
2. Poda por cota superior. Al llegar a un nodo se calcula una cota del máximo beneficio que podría obtenerse completando esa solución parcial. Si el beneficio acumulado más esa cota no supera al mejor beneficio encontrado hasta el momento, ninguna

solución de esa rama puede mejorarlo y la rama se descarta (Curso Echevarría, Teoría de Algoritmos, 2026b).

Para que la segunda poda sea correcta, la cota nunca debe subestimar el beneficio alcanzable. Se utiliza la relajación fraccional del problema: se recorren los ítems restantes incorporándose completos mientras entren, y del primero que no entra se incorpora sólo la fracción que ocupa la capacidad restante. El beneficio así obtenido es el máximo posible si se permitieran fracciones y, por lo tanto, una cota superior válida para el problema 0/1, siempre que los ítems están ordenados en forma decreciente según su relación beneficio/peso. Por ese motivo, el algoritmo ordena los ítems una única vez antes de iniciar la exploración. Este ordenamiento no altera el conjunto de soluciones exploradas, ya que en cada nodo se consideran ambas alternativas (incluir y no incluir); solo modifica el orden de recorrido.

Ese orden tiene además un efecto favorable: como en cada nodo se explora primero la rama de inclusión, el primer camino recorrido incorpora los ítems de mayor rendimiento mientras entren en la mochila. La primera solución completa encontrada es, entonces, de muy buena calidad, lo que permite que la poda por cota descarte gran parte del árbol desde el inicio.

Al finalizar, la mejor solución registrada es la óptima, ya que solo se descartaron ramas que no podían contener una solución mejor. Los índices de la solución se traducen al orden original de los ítems para devolver el mismo resultado que la fuerza bruta.

```
COTA_SUPERIOR(idx, peso_acum, pesos, beneficios, capacidad):
    cap_restante ← capacidad − peso_acum
    cota ← 0
    para i desde idx hasta n − 1:
        si pesos[i] ≤ cap_restante:
            cota ← cota + beneficios[i]
            cap_restante ← cap_restante − pesos[i]
        sino:
            cota ← cota + beneficios[i] · (cap_restante / pesos[i])
            cortar el ciclo
    retornar cota
```

```
BT(idx, peso_acum, benef_acum, seleccion, mejor, capacidad, pesos, beneficios):
   n ← longitud(pesos)

   si idx = n:                                  // caso base: solución completa
      si benef_acum > mejor.beneficio:
         mejor.beneficio ← benef_acum
         mejor.seleccion ← copia(seleccion)
      retornar

     si benef_acum + COTA_SUPERIOR(idx, peso_acum, pesos, beneficios, capacidad) ≤
mejor.beneficio:
      retornar                                  // poda por cota

   si peso_acum + pesos[idx] ≤ capacidad:       // poda por capacidad
      agregar idx a seleccion                   // rama 1: incluir el ítem idx
      BT(idx + 1, peso_acum + pesos[idx], benef_acum + beneficios[idx],
        seleccion, mejor, capacidad, pesos, beneficios)
      quitar idx de seleccion                   // retroceso

   BT(idx + 1, peso_acum, benef_acum,           // rama 2: no incluir el ítem idx
     seleccion, mejor, capacidad, pesos, beneficios)


MOCHILA_BACKTRACKING(capacidad, pesos, beneficios):
   n ← longitud(pesos)
   orden ← índices 0..n−1 ordenados por beneficios[i] / pesos[i] en forma decreciente
   pesos_ord ← pesos reordenados según orden
   beneficios_ord ← beneficios reordenados según orden

   mejor ← (beneficio: 0, seleccion: vacía)
   BT(0, 0, 0, vacía, mejor, capacidad, pesos_ord, beneficios_ord)

   items ← { orden[i] : i en mejor.seleccion }   // índices originales
   retornar mejor.beneficio, items
```

b. Estructuras de datos utilizadas

Ambos algoritmos reciben la instancia como dos listas paralelas, donde la posición i de cada una corresponde al ítem i:

Estructura	Tipo	Uso
pesos	Lista de enteros de longitud n	pesos[i] es el peso del ítem i. El acceso por índice es O(1).
beneficios	Lista de enteros de longitud n	beneficios[i] es el beneficio del ítem i. El acceso por índice es O(1).
items (resultado)	Lista de enteros	Índices originales de los ítems que forman la solución óptima.

Fuerza bruta

Estructura	Tipo	Uso
mascara	Entero (conjunto de bits)	Representa un subconjunto de ítems: el bit i indica si el ítem i está incluido. Consultar un bit (mascara & (1 << i)) es O(1), y recorrer los enteros de 0 a $2^n - 1$ enumera todos los subconjuntos sin necesidad de almacenarlos.
mejor_mascara	Entero	Máscara de la mejor solución encontrada hasta el momento.
peso_total, beneficio_total, mejor_beneficio	Enteros	Acumuladores del subconjunto en evaluación y mejor beneficio registrado.

Fuera de la entrada, la fuerza bruta solo utiliza variables escalares y la lista final de ítems, por lo que su espacio adicional es O(n).

Backtracking

Estructura	Tipo	Uso
orden	Lista de enteros de longitud n	Índices de los ítems ordenados por beneficio/peso decreciente. Se usa para construir las listas ordenadas y, al final, para traducir la solución a los índices originales.
pesos_ord, beneficios_ord	Listas de enteros de longitud n	Pesos y beneficios en el orden de orden. Se construyen juntas a partir de los mismos índices, de modo que cada peso conserva su beneficio.
seleccion	Lista utilizada como pila	Ítems incluidos en la rama que se está explorando. Al incluir un ítem se apila (append) y al retroceder se desapila (pop), ambas operaciones O(1). Se usa una única lista compartida por todas las llamadas, en lugar de crear una nueva en cada una.
mejor	Lista mutable de dos elementos: [beneficio, selección]	Mejor solución encontrada. Al ser un objeto mutable pasado por referencia, todas las llamadas recursivas leen y actualizan la misma instancia. Cuando se registra una nueva mejor solución se guarda una copia de seleccion, porque esa lista sigue modificándose durante el resto de la exploración.
Pila de llamadas recursivas	Implícita	Cada llamada activa corresponde a un nodo del camino actual en el árbol de decisión. Su profundidad máxima es n + 1.

El espacio adicional del backtracking es O(n): las listas ordenadas, seleccion y la pila de recursión tienen como máximo n + 1 elementos cada una.

1.3 Seguimientos

Se utiliza la siguiente instancia reducida, con capacidad W = 5:

Ítem	Peso	Beneficio	Beneficio/peso
0	4	5	1,25
1	2	3	1,50
2	3	4	1,33

Fuerza bruta

El algoritmo recorre las $2^3 = 8$ máscaras. El bit menos significativo corresponde al ítem 0.

Máscara	Binario	Ítems incluidos	Peso total	Beneficio total	factible	Mejor beneficio
0	000	—	0	0	Sí	0
1	001	{0}	4	5	Sí	5
2	010	{1}	2	3	Sí	5
3	011	{0, 1}	6	8	No (6 > 5)	5
4	100	{2}	3	4	Sí	5
5	101	{0, 2}	7	9	No (7 > 5)	5
6	110	{1, 2}	5	7	Sí	7
7	111	{0, 1, 2}	9	12	No (9 > 5)	7

Resultado: beneficio óptimo 7, con los ítems {1, 2} (peso 5). Se evaluaron las 8 soluciones completas, incluidas las tres infactibles.

Backtracking

Ordenamiento previo. Ordenando por beneficio/peso decreciente se obtiene el orden de recorrido: ítem 1 (1,50), ítem 2 (1,33), ítem 0 (1,25). En el árbol, el nivel 0 decide sobre el ítem 1, el nivel 1 sobre el ítem 2 y el nivel 2 sobre el ítem 0.

Exploración. En la tabla, "+k" indica que el ítem k se incluyó y "−k" que se excluyó. La columna "Cota" muestra el beneficio acumulado más la cota superior fraccional de lo que resta.

Paso	Camino	Peso acum.	Benef. acum.	Cota	Mejor	Acción
1	(raíz)	0	0	0 + 7 = 7	0	7 > 0: explora. Incluir ítem 1 (2 ≤ 5)
2	+1	2	3	3 + 4 = 7	0	7 > 0: explora. Incluir ítem 2 (5 ≤ 5)
3	+1 +2	5	7	7 + 0 = 7	0	7 > 0: explora. Incluir ítem 0: 5 + 4 = 9 > 5, poda por capacidad
4	+1 +2 −0	5	7	—	7	Hoja: 7 > 0, actualiza mejor = 7
5	+1 −2	2	3	3 + 3,75 = 6,75	7	6,75 ≤ 7: poda por cota
6	−1	0	0	0 + 6,5 = 6,5	7	6,5 ≤ 7: poda por cota

Detalle de las cotas que producen las podas:

- Paso 5: quedan 3 unidades de capacidad y solo el ítem 0 por decidir (peso 4). No entra completo, por lo que se suma la fracción 3/4 de su beneficio: 5 · 3/4 = 3,75. Aun en el mejor caso fraccional, la rama alcanzaría 6,75, que no supera al 7 ya encontrado.
- Paso 6: con capacidad 5 disponible, el ítem 2 entra completo (beneficio 4, quedan 2 unidades) y del ítem 0 entra la fracción 2/4 (beneficio 2,5). La cota es 6,5 ≤ 7, por lo que se descarta toda la rama en la que el ítem 1 no está incluido.

Resultado: beneficio óptimo 7. La selección en el orden de recorrido corresponde a las posiciones 0 y 1, que traducidas a los índices originales son los ítems {1, 2}: el mismo resultado que la fuerza bruta.

Comparación. El árbol de decisión completo tiene 15 nodos (1 + 2 + 4 + 8) y 8 hojas. El backtracking visitó 6 nodos y alcanzó una sola hoja: la primera solución completa encontrada ya era la óptima, gracias al ordenamiento por beneficio/peso, y a partir de ella la poda por cota descartó el resto del árbol.

1.4 Complejidad

Fuerza bruta

El análisis se realiza a partir del pseudocódigo. La inicialización de n, mejor_beneficio y mejor_mascara tiene costo constante. El ciclo externo recorre todas las máscaras de 0 a $2^n - 1$, es decir, realiza 2^n iteraciones. En cada una se reinician los acumuladores en O(1) y se ejecuta el ciclo interno, que recorre los n bits de la máscara; para cada bit, la consulta y la actualización de peso_total y beneficio_total cuestan O(1), por lo que el ciclo interno cuesta Θ(n). Luego, la comparación con la mejor solución y su eventual actualización cuestan O(1). Finalmente, construir la lista items a partir de mejor_mascara cuesta O(n).

El tiempo total resulta:

$T(n) = 2^n \cdot (c_1 \cdot n + c_2) + c_3 \cdot n \in \Theta(n \cdot 2^n)$

La cota es ajustada y vale para cualquier instancia: el algoritmo no tiene ningún corte anticipado, de modo que siempre recorre los 2^n subconjuntos y, para cada uno, los n bits. Su mejor caso y su peor caso coinciden.

Backtracking

Preprocesamiento y postprocesamiento. En MOCHILA_BACKTRACKING, el ordenamiento de los índices por beneficio/peso cuesta O(n log n), la construcción de pesos_ord y beneficios_ord cuesta O(n), y la traducción final de la selección a los índices originales, que incluye ordenarlos, cuesta O(n log n).

Costo de cada nodo. En cada llamada a BT, el caso base compara el beneficio y, si mejora, copia seleccion, con costo O(n). En los nodos internos, el cálculo de COTA_SUPERIOR recorre a lo sumo los n − idx ítems restantes, con costo O(n). La poda por capacidad y las operaciones agregar y quitar sobre seleccion cuestan O(1). Por lo tanto, cada nodo cuesta O(n) y el tiempo total es O(n) multiplicado por la cantidad de nodos visitados.

Peor caso. Si las podas no descartan ninguna rama, el algoritmo recorre el árbol de decisión completo. Llamando T(k) al costo de resolver un subárbol con k ítems por decidir, cada nodo realiza trabajo O(n) y dos llamadas recursivas:

$T(k) = 2 \cdot T(k - 1) + O(n)$, $T(0) = O(n)$

Desarrollando la recurrencia, el árbol tiene $1 + 2 + 4 + \ldots + 2^n = 2^{n+1} - 1$ nodos, cada uno de costo O(n), por lo que $T(n) \in O(n \cdot 2^{n+1}) = O(n \cdot 2^n)$. Sumando el preprocesamiento, $O(n \log n) + O(n \cdot 2^n) = O(n \cdot 2^n)$: el mismo orden que la fuerza bruta. Esta cota se alcanza efectivamente: en la sección 1.6 se presentan instancias en las que la poda por cota casi nunca actúa y los tiempos crecen según $n \cdot 2^n$.

Mejor caso. Si la primera solución completa encontrada, que incorpora los ítems de mayor beneficio/peso mientras entren, es la óptima, y a partir de ella la poda por cota descarta en cada nivel la rama alternativa, el algoritmo visita un único camino de n + 1 nodos más, a lo sumo, un nodo podado por nivel. Son O(n) nodos de costo O(n) cada uno, por lo que el tiempo es $O(n^2)$, más el O(n log n) del ordenamiento.

Caso real. La cantidad de nodos visitados depende de la efectividad de las podas en cada instancia, por lo que el tiempo real se ubica entre ambos extremos. Por este motivo, la complejidad del backtracking se expresa por su peor caso, $O(n \cdot 2^n)$, aclarando que en la práctica la poda reduce drásticamente el espacio explorado. La sección 1.7 contrasta estas cotas con los tiempos medidos.

1.5 Set de datos

Se utilizaron dos familias de sets de datos. Todos se encuentran en la carpeta Ej1_FB_BT/sets/ y pueden regenerarse con el script generar_sets.py.

Sets generados con crear_mochila.py

Constituyen la medición principal y se generaron con el código provisto por la cátedra, sin modificaciones. Cada archivo mochila<n>.txt contiene n ítems con pesos aleatorios entre 1 y 200 y beneficios aleatorios entre 1 y 1000, y una capacidad de 50·n, de modo que aproximadamente la mitad de los ítems entra en la mochila. Como crear_mochila.py utiliza números aleatorios, generar_sets.py fija una semilla antes de invocarlo (semilla 2026). De este modo, la generación es reproducible: ejecutar nuevamente el script produce exactamente los mismos archivos que se entregan.

Los tamaños se eligieron según el comportamiento esperado de cada algoritmo:

- Rango común, n = 4, 6, 8, …, 22 (10 sets). Se ejecutan ambos algoritmos sobre estas mismas instancias, lo que permite comparar sus tiempos y verificar que obtienen el mismo beneficio óptimo. El límite superior está dado por la fuerza bruta: con n = 22 tarda más de 6 segundos y, como cada ítem adicional duplica el tiempo, n = 24 superaría los 25 segundos y n = 26 rondaría los dos minutos, lo que vuelve impracticable la medición. Se avanza de a dos ítems porque, en un algoritmo exponencial, cada paso multiplica el tiempo aproximadamente por cuatro, lo que produce puntos equiespaciados en escala logarítmica y cubre más de seis órdenes de magnitud de tiempo.
- Rango extendido para backtracking, n = 50, 100, 200, 300, 500, 750, 1000, 1500, 2000 (9 sets). Se ejecuta solo el backtracking, ya que la fuerza bruta no puede resolver estas instancias en un tiempo razonable. El objetivo es mostrar hasta qué tamaño escala el algoritmo gracias a las podas y caracterizar el crecimiento real de sus tiempos.

Sets de peor caso

Además, se construyeron manualmente siete sets, peor_caso<n>.txt con n = 10, 12, …, 22, diseñados para que la poda por cota del backtracking sea lo menos efectiva posible. Su objetivo es verificar empíricamente que la complejidad de peor caso $O(n \cdot 2^n)$ obtenida en la

sección 1.4 efectivamente se alcanza. Se usa el mismo rango de tamaños que en la fuerza bruta para poder comparar ambos algoritmos sobre estas instancias.

Cada set se construye con tres propiedades:

- El beneficio de cada ítem es igual a su peso. Todos los ítems tienen la misma relación beneficio/peso, por lo que el ordenamiento no aporta información y la cota fraccional se reduce al beneficio acumulado más la capacidad restante.
- Todos los pesos son pares (entre 2 y 200), y la capacidad es impar (aproximadamente la mitad de la suma de los pesos).
- En consecuencia, ninguna combinación llena exactamente la mochila, porque toda suma de números pares es par. La mejor solución tiene beneficio a lo sumo $W - 1$, mientras que la cota en cada nodo es el beneficio acumulado más toda la capacidad restante. Como esa cota casi siempre supera a la mejor solución encontrada, la poda por cota casi nunca se activa y el algoritmo recorre prácticamente todo el árbol de decisión.

En total se utilizaron 26 sets de datos: 19 generados con crear_mochila.py y 7 de peor caso.

1.6 Tiempos de ejecución

Metodología de medición

Las mediciones se realizaron con el script medir.py en una mac con macOS y Python 3.9.6. Para cada set se mide únicamente el tiempo de ejecución del algoritmo, sin incluir la lectura del archivo, utilizando time.perf_counter(). Las ejecuciones que tardan menos de 0,5 segundos se repiten cinco veces y se registra el tiempo mínimo, ya que en tiempos tan pequeños la variabilidad introducida por el sistema operativo es comparable al tiempo medido; el mínimo es el valor menos afectado por esas interferencias. Los resultados de cada set (beneficio, peso utilizado, ítems elegidos y tiempo) se guardan en los archivos CSV de la carpeta resultados/.

La curva teórica de cada gráfico corresponde a la complejidad $O(n \cdot 2^n)$ obtenida en la sección 1.4, de la forma $T(n) = c \cdot n \cdot 2^n$. La constante c, que depende de la máquina y de la implementación, se ajusta por mínimos cuadrados sobre el logaritmo de los tiempos

medidos. Para el ajuste se descartan los tiempos menores a 10^{-4} segundos, porque en ese rango el costo fijo de la llamada a la función domina sobre el trabajo del algoritmo.

Varios gráficos utilizan escala logarítmica en el eje vertical. En esa escala, una función exponencial se ve como una recta, lo que permite comparar en un mismo gráfico tiempos que van desde microsegundos hasta segundos.

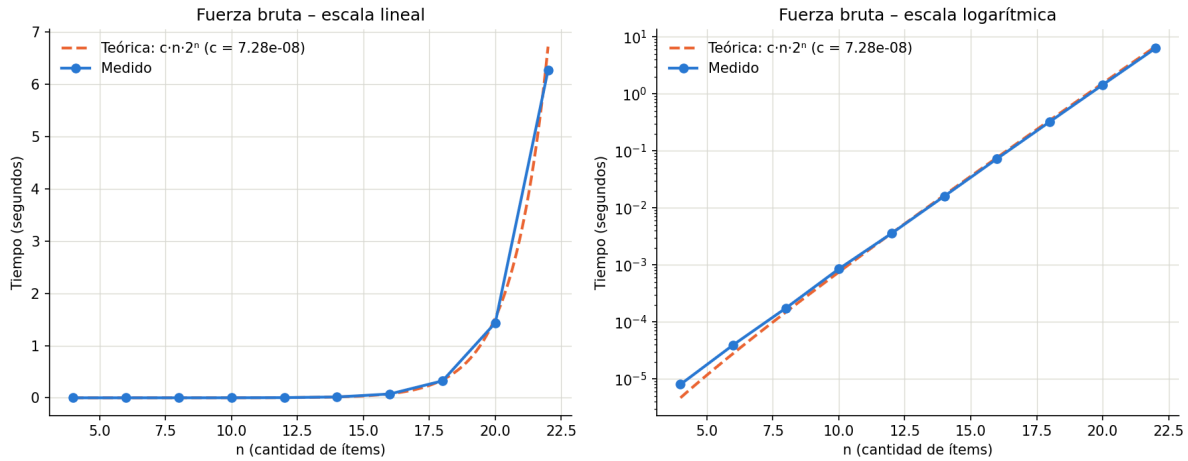


Figura 1. Fuerza bruta: tiempos medidos y curva teórica $c \cdot n \cdot 2^n$, en escala lineal (izquierda) y logarítmica (derecha).

En escala lineal se observa el crecimiento exponencial característico: los tiempos son despreciables hasta $n \approx 14$ y luego se disparan, pasando de 1,43 segundos en $n = 20$ a 6,27 segundos en $n = 22$. En escala logarítmica, los puntos medidos se ubican prácticamente sobre la recta teórica en todo el rango, desde 10^{-5} hasta 10^{1} segundos, con una constante ajustada $c \approx 7{,}28 \cdot 10^{-8}$ segundos. Cada incremento de dos ítems multiplica el tiempo aproximadamente por 4,4, valor cercano al factor teórico $4 \cdot (n + 2)/n$. Se concluye que los tiempos de la fuerza bruta se corresponden con la complejidad $\Theta(n \cdot 2^n)$

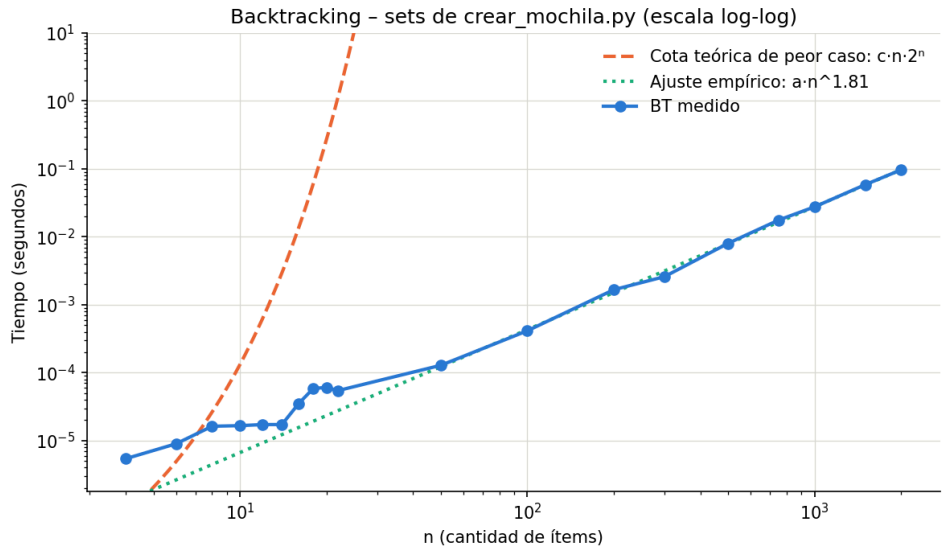


Figura 2. Backtracking sobre los sets de crear_mochila.py: tiempos medidos, cota teórica de peor caso c·n·2ⁿ y ajuste empírico a·n^k, en escala log-log.

Con los sets de la cátedra, el backtracking resuelve instancias de hasta 2000 ítems en menos de 0,1 segundos. La curva de peor caso, trazada con la constante ajustada sobre los sets de peor caso, crece tan rápido que abandona el gráfico antes de n = 30, mientras que los tiempos medidos permanecen varios órdenes de magnitud por debajo. Para describir el comportamiento observado se ajustó una curva de la forma a·n^k sobre los sets con n ≥ 50, obteniéndose k ≈ 1,81. Este exponente es menor que 2, lo que indica que en estas instancias el algoritmo se comporta de forma cercana a su mejor caso O(n²): la cantidad de nodos visitados crece aproximadamente de forma lineal con n, y cada nodo cuesta O(n) por el cálculo de la cota. Para n < 8, los tiempos medidos quedan por encima de la curva de peor caso porque, con tan pocos ítems, domina el costo fijo del ordenamiento y de las llamadas, y no la exploración del árbol.

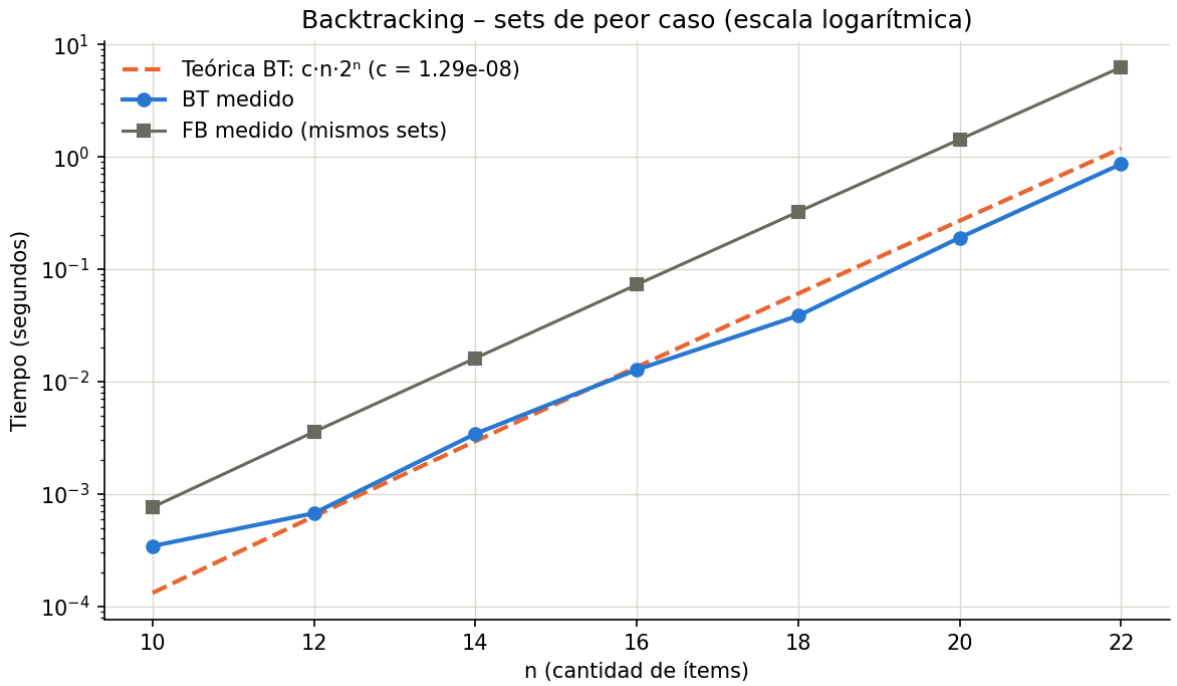


Figura 3. Backtracking sobre los sets de peor caso: tiempos medidos, curva teórica $c \cdot n \cdot 2^n$ y tiempos de la fuerza bruta sobre los mismos sets, en escala logarítmica.

Sobre los sets de peor caso, los tiempos del backtracking siguen la recta teórica $c \cdot n \cdot 2^n$, con $c \approx 1{,}29 \cdot 10^{-8}$ segundos: a partir de n = 18, cada incremento de dos ítems multiplica el tiempo por un factor de entre 4,5 y 4,9, el mismo comportamiento que la fuerza bruta. Ambos algoritmos resultan rectas paralelas en escala logarítmica, es decir, tienen el mismo orden de crecimiento y solo difieren en una constante: en n = 22, el backtracking tarda 0,86 segundos y la fuerza bruta 6,26 segundos. Esta diferencia constante se debe a la poda por capacidad, que sigue descartando las ramas infactibles aun cuando la poda por cota no actúa. Se concluye que la cota $O(n \cdot 2^n)$ del peor caso se alcanza efectivamente.

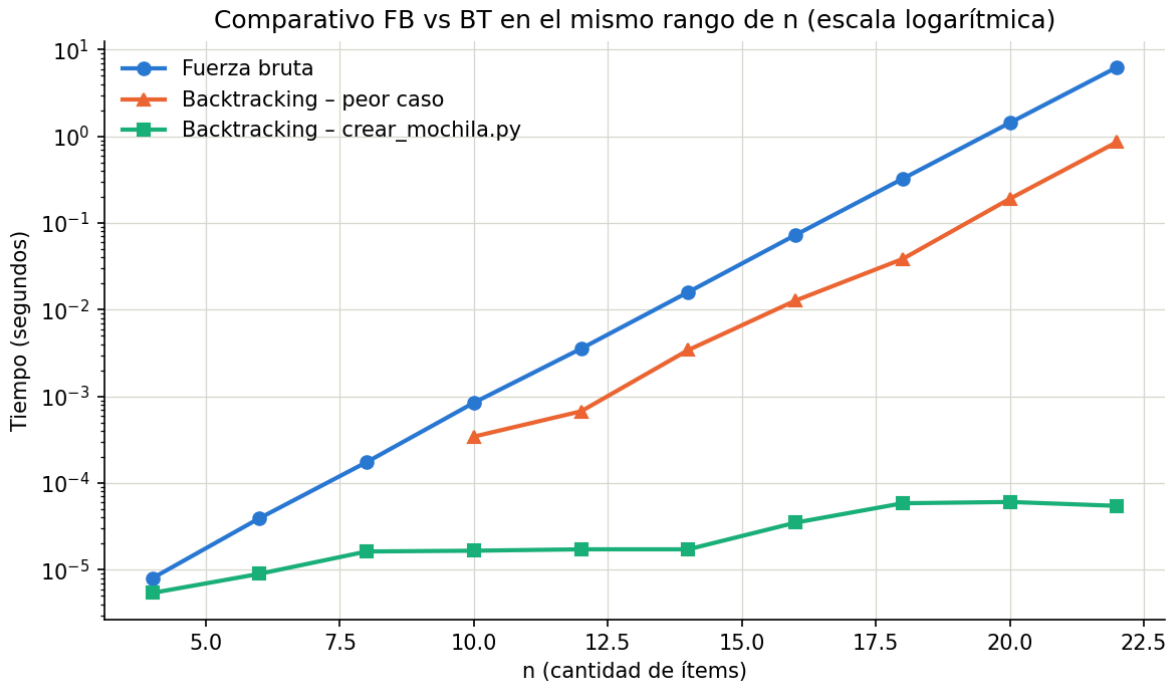


Figura 4. Comparación de la fuerza bruta y el backtracking en el rango común n = 4 a 22, en escala logarítmica.

El gráfico reúne las tres series en el rango en que la fuerza bruta puede ejecutarse. La fuerza bruta y el backtracking en peor caso crecen como rectas paralelas, mientras que el backtracking sobre los sets de crear_mochila.py se mantiene prácticamente constante, por debajo de 10^{-4} segundos. En n = 22, sobre la misma instancia, la fuerza bruta tarda 6,27 segundos y el backtracking 0,000055 segundos: más de cien mil veces menos. La brecha entre ambos algoritmos crece exponencialmente con n, porque la fuerza bruta evalúa los 2^n subconjuntos mientras que el backtracking explora solo una pequeña fracción del árbol.

1.7 Informe de Resultados

En los 17 sets en los que corrimos ambos algoritmos (los 10 de crear_mochila.py del rango común y los 7 de peor caso), fuerza bruta y backtracking obtuvieron el mismo beneficio óptimo. Esto queda registrado en los archivos verificacion_tipicos.txt y verificacion_peor_caso.txt de la carpeta resultados/. Como la fuerza bruta evalúa todas las soluciones, su resultado es siempre el óptimo, así que la coincidencia muestra que las podas del backtracking solo descartan ramas que no podían mejorar la solución. Es decir, el backtracking llega al mismo resultado que la fuerza bruta, que es lo que pide el enunciado.

En el caso de la fuerza bruta, los tiempos medidos se corresponden con la complejidad calculada. Los puntos siguen la curva $c \cdot n \cdot 2^n$ en todo el rango medido (Figura 1). Era lo esperable, porque el algoritmo recorre los 2^n subconjuntos sin importar los valores de la instancia. Por lo mismo, cada ítem que se agrega duplica el tiempo, y con 22 ítems ya tarda más de 6 segundos.

Para el backtracking hay que separar dos situaciones. Con los sets de peor caso, los tiempos también siguen la curva $c \cdot n \cdot 2^n$ y crecen en paralelo a los de la fuerza bruta (Figuras 3 y 4). Esto confirma que la cota $O(n \cdot 2^n)$ de la sección 1.4 se alcanza en la práctica: el backtracking no mejora el orden de complejidad en el peor caso. Con los sets de crear_mochila.py, en cambio, los tiempos quedan muy por debajo de esa cota. Resuelve 2000 ítems en menos de 0,1 segundos y el crecimiento observado se ajusta a n^1,81 (Figura 2), menos que el $O(n^2)$ del mejor caso. Esto no contradice el análisis, porque $O(n \cdot 2^n)$ es un límite superior y en estas instancias las podas descartan casi todo el árbol.

La diferencia entre ambos tipos de sets está en cómo funciona la cota. En los sets de la cátedra los ítems tienen relaciones beneficio/peso muy distintas entre sí. Al recorrerlos ordenados por esa relación, el primer camino ya arma una solución muy cercana a la óptima y la cota fraccional queda muy ajustada, así que casi cualquier rama alternativa se poda. En los sets de peor caso todos los ítems tienen la misma relación beneficio/peso: ordenarlos no sirve de nada y la cota siempre queda por encima de la mejor solución, por lo que casi nunca poda. O sea, que el backtracking ande bien o mal depende de cómo son los datos y no solo de cuántos ítems hay.

Comparando directamente, sobre la misma instancia de 22 ítems la fuerza bruta tarda 6,27 segundos y el backtracking 0,000055 segundos, y la diferencia se agranda a medida que crece n. Incluso en el peor caso el backtracking es unas siete veces más rápido, por la poda por capacidad, aunque crece al mismo ritmo.

Como conclusión, los dos algoritmos encuentran siempre la solución óptima y los dos son exponenciales en el peor caso. La fuerza bruta es más simple y su tiempo es totalmente predecible, pero solo sirve para instancias de unos 20 ítems. El backtracking, gracias a las podas por capacidad y por cota, explora una parte muy chica del árbol y resuelve en fracciones de segundo instancias de miles de ítems como las de crear_mochila.py, aunque su peor caso siga siendo $O(n \cdot 2^n)$.

2. Greedy

El algoritmo Greedy para el problema de la mochila busca una solución de manera constructiva haciendo en cada paso la elección localmente óptima, asi tratando de llegar a un resultado óptimo. Este criterio greedy consiste en ordenar todos los elementos en forma decreciente según su densidad de valor o rendimiento relativo, definido como el cociente entre beneficio y peso:

$$rendimiento = vi / wi$$

El algoritmo recorre la lista ordenada e incluye cada item que queda en la capacidad restante de la mochila. Como esta heurística puede fallar ante un elemento individual de gran beneficio que no es ingresado por la capacidad, utilizamos la garantía ½, esta garantía dos candidatos a solución:

1. La solución obtenida acumulando elementos a través de su rendimiento
2. El mejor elemento individual, con mayor beneficio

Al retornar el maximo entre ambos beneficios, se garantiza teóricamente que la solución obtenida aportará como mínimo la mitad del beneficio óptimo global.

2.1 Supuestos

Para el algoritmo de greedy, tenemos los siguientes supuestos:

- La capacidad de la mochila debe ser un valor entero positivo
- Los pesos y beneficios de los elementos deben ser enteros y positivos
- Los elementos no pueden ser divisibles, (problema de tipo 0/1)
- Los elementos deben ser al menos 1

2.2 a. Pseudocódigo Greedy

```
Algoritmo Lectura_y_Ordenamiento(nombre_archivo):
    Abrir nombre_archivo en modo lectura
    capacidad <- Leer primera línea como entero
    elementos <- Lista vacía
    Para cada linea restante en el archivo:
        (peso, beneficio) <- Extraer valores de la linea
        Agregar (peso, beneficio) a la lista elementos
    elementos_ordenados <- Ordenar elementos según (beneficio / peso) descendente
    Retornar capacidad, elementos_ordenados

Algoritmo Mochila_Greedy(capacidad, elementos_ordenados):
    peso_actual <- 0
    beneficio_total <- 0

    Para cada (peso, beneficio) en elementos_ordenados:
        Si peso_actual + peso <= capacidad entonces:
            peso_actual <- peso_actual + peso
            beneficio_total <- beneficio_total + beneficio

    mejor_elemento_individual <- 0

    Para cada (peso, beneficio) en elementos_ordenados:
        Si peso <= capacidad entonces:
            Si beneficio > mejor_elemento_individual entonces:
                mejor_elemento_individual <- beneficio

Retornar Máximo(beneficio_total, mejor_elemento_individual)
```

b. Estructuras de datos utilizadas

- elementos: (Lista de tuplas) almacena los elementos leídos del archivo
- elementos_ordenados (Lista de tuplas) almacena los elementos reordenados por su relación valor peso
- peso_actual, beneficio_total (int/floats) acumuladores del peso consumido y el beneficio obtenido
- mejor_elemento_individual (int/float) mantiene el maximo beneficio entre los elementos

2.3 Seguimientos

Ejemplo de seguimiento con capacidad = 10

Elem	Peso	Beneficio	B/P	Acción	Peso acum	Beneficio acum	¿Entra?
3	4	40	10	Evaluar elem	4	40	SI: 4 <= 10
1	7	42	6	Evaluar elem	4	40	No: 4 + 7 >10
4	5	25	5	Evaluar elem	9	65	Si: 4+ 5<=10
2	3	12	4	Evaluar elem	9	65	No: 9 + 3 >10

Verificación de garantía ½

Elem	Peso	Beneficio	¿Entra?	Mejor elem
3	4	40	Si: 4 <= 10	40
1	7	42	Si 7 <= 10	42

4	5	25	Si 5 <= 10	42
2	3	12	Si 3 <= 10	42

Ordenamiento inicial**:** Se ordenan los elementos en forma decreciente por su densidad B/P

Recorrido:

- Se evalúa el Ítem 3 ($w = 4 \leq 10$): entra en la mochila. Peso acumulado = 4, Beneficio acumulado = 40.
- Se evalúa el Ítem 1 ($w = 7$): no entra ($4 + 7 = 11 > 10$), por lo que se descarta.
- Se evalúa el Ítem 4 ($w = 5 \leq 10 - 4$): entra en la mochila. Peso acumulado = 9, Beneficio acumulado = 65.
- Se evalúa el Ítem 2 ($w = 3$): no entra ($9 + 3 = 12 > 10$), se descarta.
- Se obtiene $B_{greedy} = 65$.

Cálculo del mejor elemento individual:

- Recorre la lista verificando los elementos que caben por sí solos ($w_i \leq 10$). El mayor beneficio es $v_{max} = 42$ (Ítem 1).

Resolución final: El algoritmo retorna $\max(B_{greedy}, v_{max}) = \max(65, 42) = 65$.

Sea $OPT_{0/1}$ el beneficio de la solución óptima del problema de la mochila 0/1 (algoritmo implementado), y OPT_{frac} el beneficio óptimo de la relajación fraccional del problema (donde se permite tomar fracciones de ítems). Es un hecho conocido que la solución fraccional acota superiormente a la 0/1:

$$OPT_{0/1} \leq OPT_{frac}$$

Como los elementos se ordenan de la siguiente forma: v1 / w1 >= v2 / w2 >= … >= vn / wn
El algoritmo greedy fraccional toma los primeros k - 1 elementos hasta el elemento k que no cabe completamente, por lo tanto:

$$OPT_{frac} = \sum_{i=1}^{k-1} v_i + \alpha \cdot v_k$$

Donde $0 < \alpha < 1$ es la fracción del elemento k que completa la capacidad W.

Denotemos por B_{greedy} al beneficio obtenido por el algoritmo greedy implementado

$$B_{greedy} \geq \sum_{i=1}^{k-1} v_i$$

Consideramos el elemento k al primer elemento descartado, por no entrar en la mochila, su beneficio individual v_k es un candidato para v_{max} .

$$v_{max} \geq v_k$$

Sumando B_{greedy} y v_{max}:

$$B_{greedy} + v_{max} \geq \sum_{i=1}^{k-1} v_i + v_k > \sum_{i=1}^{k-1} v_i + \alpha \cdot v_k = OPT_{frac} \geq OPT_{0/1}$$

Por ende:

$$B_{greedy} + v_{max} > OPT_{0/1}$$

Como el algoritmo retorna $ALG = \max(B_{greedy}, v_{max})$, se cumple que:

$$2 \cdot ALG = 2 \cdot \max(B_{greedy}, v_{max}) \geq B_{greedy} + v_{max} > OPT_{0/1}$$

Dividiendo por 2, obtenemos la cota de garantía de aproximación:

$$ALG \geq 1/2\ OPT_{0/1}$$

Caso Borde:

El ejemplo anterior muestra una instancia donde el algoritmo alcanza el óptimo, sin embargo, es necesario presentar un contraejemplo del peor caso. Consideremos una mochila de capacidad W = M (M número entero muy grande) y n = 3 elementos:

- Elemento 1: peso1 = 1, beneficio1 = 2, rendimiento v1 / w1 = 2
- Elemento 2: peso2 = M, beneficio2 = M, rendimiento v2 / w2 = 1
- Elemento 3: peso3 =M, beneficio3 = M, rendimiento v3 / w3 = 1

Cuando se ejecuta el algoritmo, ordena los elementos y quedan de la siguiente forma

1. Elemento 1
2. Elemento 2
3. Elemento 3

Selecciona el elemento 1, con un beneficio de 2 y cuando evalúa el elemento 2 o 3, no los puede ingresar ya que M + 1 > M, por lo tanto nos pasamos de la capacidad.

Luego evalúa el mejor elemento individual $v_{max} = \max(2, M, M) = M$.

El algoritmo retorna $ALG = \max(B_{greedy}, v_{max}) = \max(2, M) = M$

Esto demuestra formalmente que la cota de $1/2$ es ajustada y que el algoritmo Greedy modificado no puede garantizar un factor de aproximación superior a $1/2$.

2.4 Complejidad

Análisis temporal:

1. **Construcción de la lista con sus rendimientos**: Recorrer los N elementos para calcular su rendimiento toma tiempo 0(N)
2. **Ordenamiento**: Ordenar los N elementos según su rendimiento toma O(N log N)
3. **Bucle Greedy**: Recorrido lineal sobre los N elementos ordenados esto toma tiempo de O(N)
4. **Bucle de mejor individual**: También toma O(N) ya que recorre la lista para encontrar el mejor elemento individual
5. **Comparacion final**: Esto es una operación que toma O(1)

Complejidad temporal total:

$$T(n) = O(n \log n) + \Theta(n) + \Theta(n) = O(n \log n)$$

Dado que la etapa dominante es el ordenamiento, la complejidad global del algoritmo es O(N log N).

Análisis espacial:

- La lista auxiliar de elementos con sus rendimientos requiere almacenar n tuplas de tamaño cte por lo que ocupa espacio O(N)
- La lista de selección devuelta almacena como maximo n indices, requiriendo O(N)

Complejidad espacial total:

$$S(n) \in O(n)$$

2.5 Set de datos

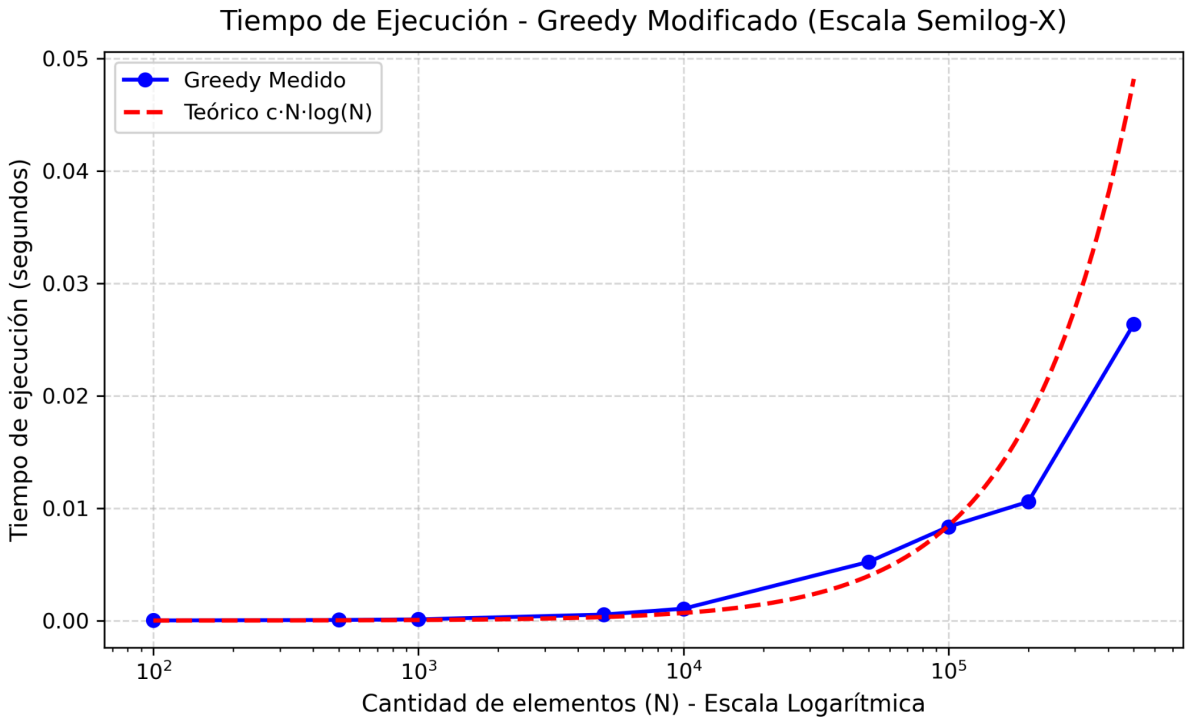
Para evaluar el rendimiento empírico del algoritmo y verificar que los tiempos de ejecución responden a la cota teórica de O(N log N), se generaron instancias de prueba escaladas utilizando el generador (crear_mochila.py).

A diferencia de los algoritmos como fuerza bruta o backtracking, la heurística greedy procesa volúmenes masivos de datos en fracciones de segundo. Por este motivo se evaluó un rango extendido de datos hasta N = 500000.

- Rango de instancias (N): 100, 500, 1000, 5000, 10000, 50000, 100000, 200000, 500000
- Para cada valor de N, se genero un archivo “mochila_N.txt” con capacidad 50 x N, pesos entre 1 y 200 y beneficios entre 1 y 1000

2.6 Tiempo de ejecución

En el eje x se presentan los tiempos de ejecución de función de los elementos N, graficada de tal forma que se pueda notar la evolución desde instancias pequeñas hasta instancias de medio millón de elementos



2.7 Informe de Resultados

1. La curva teórica $T(n) = c \cdot n\log_2(n)$ fue ajustada por mínimos cuadrados sobre las instancias con $N \geq 10.000$, obteniendo una constante proporcionalidad $c \approx 1,78 \cdot 10^{-8}$ segundos. La superposición entre la serie empírica y la curva teórica confirma que el orden de crecimiento del algoritmo está estrictamente dominado por el ordenamiento inicial O(N log N)
2. Las mediciones demuestran la altísima eficiencia de la estrategia greedy, resolver una instancia de N = 500000 requiere apenas unos pocos segundos
3. En todas las instancias evaluadas provenientes de crear_mochila.py, el beneficio alcanzado por greedy representó más del 98% del valor óptimo global. Esto evidencia que si bien la garantía formal del algoritmo en el pero caso es del 50%, en instancias aleatorias la heurística ofrece soluciones casi óptimas en tiempos insignificantes

3. Programación dinámica

3.1 Supuestos

Los algoritmos que se presentan en esta sección son correctos bajo las condiciones que siguen.

- **Elección:** Cada elemento se toma completo o no se toma, y como máximo una vez. Esto permite que la recurrencia tenga solo dos opciones por elemento, incluirlo o descartarlo.
- **Pesos y beneficios son enteros positivos** ($p_i \geq 1$ y $v_i \geq 1$). Los valores se usan como índices de la tabla, por ende deben ser enteros. Que sean positivos garantiza que cada elemento incluido reduce estrictamente la capacidad (o el beneficio) restante, de modo que la reconstrucción siempre termina. En los sets utilizados el peso va de 1 a 200 y el beneficio de 1 a 1000.
- **Capacidad W y beneficio objetivo B enteros no negativos.** Estos dos son los parámetros que fijan el ancho de la tabla de cada planteo.
- **Elementos que no entran.** Se admiten elementos con $p_i > W$, aunque los mismos nunca forman parte de la solución, ya que la recurrencia los descarta cuando su peso supera la capacidad disponible.
- **Formato de los archivos de entrada.** Aquel que genera crear_mochila.py: la primera línea es la capacidad W y cada línea siguiente es un par peso,beneficio. En los algoritmos, los elementos son una lista de tuplas (peso, beneficio).
- **Beneficio fijo significa exactamente B (planteo alternativo).** Si ninguna combinación suma exactamente B, el algoritmo informa que no hay solución. Se eligió esta lectura porque con "como máximo B" el problema es trivial (la mochila vacía tiene peso 0). En los experimentos se puede tomar B igual al beneficio óptimo del planteo tradicional para el mismo set, por lo que en ese caso se sabe de antemano que será alcanzable.
- **Varias soluciones óptimas.** La reconstrucción devuelve una sola. Ante un empate, no incluye el elemento de mayor índice.
- **Memoria.** La tabla completa se guarda en memoria, porque la reconstrucción necesita consultar todas sus filas. Esto acota el tamaño de W y de B que se puede resolver (ver 3.4).
- **Algoritmos pseudopolinomiales.** El tiempo depende del valor numérico de W (o de B) y no solo de la cantidad de elementos n

3.2 Diseño

Idea general

La programación dinámica resuelve un problema combinando las soluciones de subproblemas más chicos que se repiten, y guarda cada resultado en una tabla como memoria para calcularlo una sola vez. Sirve cuando el problema tiene subestructura óptima, ya que la solución óptima contiene soluciones óptimas de subproblemas (Cormen et al., 2022).

En el problema de la mochila tenemos que decidir, para cada elemento i, si se lo incluye o no. Si no se lo incluye, queda el mismo problema con i − 1 elementos y la misma capacidad. Si se lo incluye, queda el problema con i − 1 elementos y la capacidad reducida en p_i (que es el peso del elemento incluido), más el beneficio v_i que se gana. La solución óptima es la mejor de las dos opciones.

Por su parte la fuerza bruta explora los 2^n subconjuntos, es decir que probaría las dos alternativas por cada elemento. Pero los subproblemas "los primeros i elementos con capacidad w" son solo (n + 1)·(W + 1) y se repiten muchas veces por lo que guardarlos en una tabla evita tener que recalcularlos y baja el costo de exponencial a Θ(n·W).

Se implementaron dos planteos con la misma estructura. Se diferencian en qué magnitud se fija y cuál se optimiza:

	Planteo Tradicional	Planteo Alternativo
Magnitud fija (dato)	Capacidad W	Beneficio B
Magnitud que se optimiza	Beneficio, se maximiza	peso, se minimiza
Subproblema	M[i][w]	P[i][b]
Columnas de la tabla	w = 0, 1, …, W	b = 0, 1, …, B
Respuesta	M[n][W]	P[n][B]
Costo	Θ(n·W)	Θ(n·B)

3.2.1 Planteo tradicional: maximizar el beneficio para un peso fijo W:

Qué busca. El mayor beneficio total que se puede lograr con un subconjunto de elementos cuyo peso total no supere W.

Subproblema. M[i][w] es el mayor beneficio que se logra usando solo los primeros i elementos con una capacidad w.

Ecuación de recurrencia:

$$\mathrm{M[i][w]} = \begin{cases} M[i-1][w] & \text{si } p_i > w \\ \max\left(M[i-1][w],\ v_i + M[i-1][w-p_i]\right) & \text{si } p_i \leq w \end{cases}$$

(Se adjunta una imágen de la ecuación hecha en formato LaTex para mayor comprensión de los símbolos)

Explicación. Sea S una solución óptima para el subproblema (i, w).

1. Si $p_i > w$, el elemento i no entra, así que S usa solo los primeros i − 1 elementos y su beneficio es M[i−1][w].
2. Si $p_i \leq w$ y S no contiene al elemento i, el beneficio es M[i−1][w].
3. Si $p_i \leq w$ y S contiene al elemento i, entonces S sin el elemento i es una solución para (i − 1, w − p_i). Tiene que ser óptima para ese subproblema: si existiera una mejor, reemplazarla en S daría una solución mejor que S, lo cual es absurdo. Por eso el beneficio de S es v_i + M[i−1][w − p_i].

Como S es óptima, su beneficio es el mayor de los casos posibles, y eso es la recurrencia.

Casos base. M[0][w] = 0, porque sin elementos no hay beneficio. M[i][0] = 0, porque con pesos positivos ningún elemento entra en capacidad 0; la recurrencia ya lo produce.

Orden de llenado. Fila por fila, con i creciente. Cada celda depende solo de la fila anterior, que ya está calculada.

Reconstrucción. El valor M[n][W] dice cuánto beneficio se logra, pero no con qué elementos. Se recorre la tabla desde (n, W) hacia atrás. Como M[i][w] ≥ M[i−1][w] siempre, si ambos valores son distintos el máximo lo alcanzó la opción "incluir", así que el elemento i

está en la solución y se pasa a la capacidad w − p_i. Si son iguales, existe una solución óptima sin el elemento i y se sigue con la misma capacidad.

Qué se consigue. El beneficio óptimo exacto y un subconjunto que lo logra, en tiempo Θ(n·W). A diferencia de un greedy, ésto no es una aproximación.

Pseudocódigo:

```
Mochila_max_beneficio(elementos, W)
    // elementos[1..n], cada uno es (peso, beneficio), leer referencias
    n ← cantidad de elementos
    M ← matriz de (n+1) filas y (W+1) columnas, toda en 0

    para i desde 1 hasta n hacer
        (peso_i, beneficio_i) ← elementos[i]
        para w desde 0 hasta W hacer
            si peso_i > w entonces
                M[i][w] ← M[i-1][w]
            sino
                no_incluir ← M[i-1][w]
                incluir ← beneficio_i + M[i-1][w - peso_i]
                M[i][w] ← máximo(no_incluir, incluir)

    // reconstrucción de la solución
    elegidos ← lista vacía
    w ← W
    para i desde n hasta 1 (de a -1) hacer
        si M[i][w] ≠ M[i-1][w] entonces
            agregar elementos[i] a elegidos
            w ← w - peso de elementos[i]

    devolver M[n][W], invertir(elegidos)
```

Nota:

En la implementación la tabla se guarda transpuesta (matriz[w][i], con las capacidades como filas), lo que no cambia el algoritmo ni su complejidad.

3.2.2 Planteo alternativo: minimizar el peso con un beneficio fijo B

Qué busca. El menor peso total con el que se puede lograr un beneficio exactamente igual a B.

Subproblema. P[i][b] es el menor peso necesario para lograr un beneficio exactamente igual a b usando solo los primeros i elementos. Si ese beneficio no se puede lograr, P[i][b] vale ∞. El ∞ nos permite distinguir un "no se puede" de cualquier peso real, porque es mayor que todos y cumple ∞ + p = ∞.

Ecuación de recurrencia:

$$\mathrm{P[i][b]} = \begin{cases} P[i-1][b] & \text{si } v_i > b \\ \min\left(P[i-1][b],\ p_i + P[i-1][b-v_i]\right) & \text{si } v_i \leq b \end{cases}$$

(Se adjunta una imágen de la ecuación hecha en formato LaTex para mayor comprensión de los símbolos)

Explicación. Es el mismo razonamiento que en el planteo tradicional, con los roles de peso y beneficio intercambiados. Si $v_i > b$, el elemento i aporta más beneficio del que se necesita y no puede formar parte de una solución exacta. Si $v_i \leq b$, una solución óptima o no usa el elemento i (peso P[i−1][b]) o lo usa, y entonces el resto es una solución de peso mínimo para el beneficio $b - v_i$ con los primeros $i - 1$ elementos (peso P[i−1][b − v_i]), al que se le suma el peso p_i. Se queda con el menor de los dos.

Casos base. P[i][0] = 0 para todo i, porque lograr beneficio 0 no requiere llevar nada. P[0][b] = ∞ para b > 0, porque sin elementos no se logra ningún beneficio positivo.

Orden de llenado. Fila por fila, con i creciente, igual que en el planteo tradicional.

Reconstrucción. Como P[i][b] = mín(P[i−1][b], …) nunca es mayor que P[i−1][b], si los dos valores son distintos el primero es estrictamente menor y se logró usando el elemento i. Entonces se incluye el elemento i y se pasa al beneficio b − v_i. Si son iguales, existe una solución óptima sin ese elemento. Antes de reconstruir se verifica que P[n][B] ≠ ∞; si lo es, no hay solución.

Qué se consigue. El peso mínimo exacto para el beneficio B y un subconjunto que lo logra, en tiempo Θ(n·B).

Pseudocódigo:

```
Mochila_min_peso(elementos, B)
    n ← cantidad de elementos
    INF ← infinito
    P ← matriz de (n+1) filas y (B+1) columnas, toda en INF
    para i desde 0 hasta n hacer
        P[i][0] ← 0

    para i desde 1 hasta n hacer
        (peso_i, beneficio_i) ← elementos[i]
        para b desde 1 hasta B hacer
            si beneficio_i > b entonces
                P[i][b] ← P[i-1][b]
            sino
                no_incluir ← P[i-1][b]
                incluir ← peso_i + P[i-1][b - beneficio_i]
                P[i][b] ← mínimo(no_incluir, incluir)

    si P[n][B] = INF entonces
        devolver "no existe"

    // reconstrucción de la solución
    elegidos ← lista vacía
    b ← B
    para i desde n hasta 1 (de a -1) hacer
        si P[i][b] ≠ P[i-1][b] entonces
            agregar elementos[i] a elegidos
            b ← b - beneficio de elementos[i]

    devolver P[n][B], invertir(elegidos)
```

Estructuras de datos utilizadas en los algoritmos:

Referencia	Estructura de datos	Uso
elementos	Lista de tuplas (peso, beneficio)	Almacenamiento de los datos leídos del archivo
M / P	Lista de listas	Tabla de subproblemas
INF	Constante (infinito; en el código N)	Marca el beneficio inalcanzable en P. Es mayor que cualquier peso. Se usa en el planteo alternativo.
n	Entero	Cantidad de elementos
peso_i, beneficio_i	Enteros	Peso y beneficio i del elemento que se esta evaluando
no_incluir, incluir	Enteros	Beneficio de la celda si no se elige/si se elige el elemento i
elegidos	Lista de tuplas (peso, beneficio)	Elementos de la solución, armados en la reconstrucción
w	Entero	Capacidad en curso, recorre la tabla y, en la reconstrucción, lo que queda de capacidad
b	Entero	Beneficio en curso, recorre la tabla y, en la reconstruccion, lo que falta por lograr

Relación entre los dos planteos:

Los dos planteos resuelven el mismo problema visto desde lados opuestos. Sea B* = M[n][W] el beneficio óptimo del planteo tradicional. Entonces P[n][B*] ≤ W y P[n][b] > W para todo b > B*. Esto quiere decir que B* es el mayor b para el cual el peso mínimo no supera la capacidad.

Por qué sucede esto: Existe un subconjunto de beneficio B* y peso a lo sumo W, por lo tanto P[n][B*] ≤ W. Si hubiera un b > B* con P[n][b] ≤ W, existiría un subconjunto de beneficio mayor que B* que también cabe en la mochila, lo que contradice que B* sea el óptimo.

Esta relación justifica dos decisiones del trabajo. Primero, en los experimentos se admite la posibilidad de tomar B = B*, que garantiza que el beneficio objetivo sea alcanzable. Segundo, sirve como verificación cruzada: con B = B*, el peso mínimo que devuelve el planteo alternativo tiene que ser menor o igual que W.

3.2.3 Seguimientos

Se usa un set de 4 elementos y capacidad W = 5. Con estos datos, los dos planteos llegan a la misma solución: los elementos 1 y 2, con beneficio 7 y peso 5.

Elementos:

i	Peso	Beneficio
1	2	3
2	3	4
3	4	5
4	5	6

Planteo tradicional (W = 5)

Cada fila i agrega un elemento a los disponibles y cada columna w es una capacidad. La celda M[i][w] responde cuál es el mayor beneficio si solo se puede usar los primeros i elementos y la mochila aguanta w. La fila 0 y la columna 0 son ceros porque no hay elementos o no hay capacidad.

i \ w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

Algunas celdas explicadas:

- M[1][1] = 0. El elemento 1 pesa 2 y no entra en capacidad 1, así que se copia M[0][1] = 0.
- M[1][2] = máx(M[0][2], 3 + M[0][0]) = máx(0, 3) = 3. El elemento 1 justo entra y conviene incluirlo.
- M[2][3] = máx(M[1][3], 4 + M[1][0]) = máx(3, 4) = 4. Con capacidad 3 conviene reemplazar el elemento 1 por el 2.
- M[2][5] = máx(M[1][5], 4 + M[1][2]) = máx(3, 4 + 3) = 7. Con capacidad 5 entran los dos elementos juntos, y es la mejor combinación.
- M[4][5] = máx(M[3][5], 6 + M[3][0]) = máx(7, 6) = 7. El elemento 4 solo no mejora lo que ya se tenía.

El beneficio óptimo es M[4][5] = 7. Para saber qué elementos lo logran se reconstruye desde w = 5:

1. i = 4: M[4][5] = M[3][5] = 7, no se incluye el elemento 4.
2. i = 3: M[3][5] = M[2][5] = 7, no se incluye el elemento 3.
3. i = 2: M[2][5] = 7 ≠ M[1][5] = 3, se incluye el elemento 2 (peso 3, beneficio 4) y w pasa a 5 − 3 = 2.
4. i = 1: M[1][2] = 3 ≠ M[0][2] = 0, se incluye el elemento 1 (peso 2, beneficio 3) y w pasa a 0.

La solución es {1, 2}: peso 2 + 3 = 5 (cabe justo) y beneficio 3 + 4 = 7.

Planteo Alternativo (B = 7)

Se toma B = 7, el beneficio óptimo del planteo anterior. Ahora las columnas son beneficios y la celda P[i][b] responde cuál es el menor peso con el que se logra exactamente el beneficio b usando los primeros i elementos. Un ∞ significa que ese beneficio no se puede lograr con esos elementos. Por ejemplo, ningún subconjunto da beneficio 1 o 2, porque el beneficio mínimo de un elemento es 3.

i \ b	0	1	2	3	4	5	6	7
0	0	∞	∞	∞	∞	∞	∞	∞
1	0	∞	∞	2	∞	∞	∞	∞
2	0	∞	∞	2	3	∞	∞	5
3	0	∞	∞	2	3	4	∞	5
4	0	∞	∞	2	3	4	5	5

Algunas celdas explicadas:

- P[1][3] = mín(P[0][3], 2 + P[0][0]) = mín(∞, 2) = 2. El beneficio 3 se logra con el elemento 1, que pesa 2.
- P[2][4] = mín(P[1][4], 3 + P[1][0]) = mín(∞, 3) = 3. El beneficio 4 se logra con el elemento 2, que pesa 3.
- P[2][7] = mín(P[1][7], 3 + P[1][3]) = mín(∞, 3 + 2) = 5. El beneficio 7 se logra con los elementos 1 y 2.
- P[4][7] = mín(P[3][7], 5 + P[3][1]) = mín(5, ∞) = 5. El elemento 4 no sirve para llegar a 7, porque dejaría un resto de beneficio 1 que no se puede lograr.

El peso mínimo para beneficio 7 es P[4][7] = 5. Reconstrucción desde b = 7:

1. i = 4: P[4][7] = P[3][7] = 5, no se incluye el elemento 4.
2. i = 3: P[3][7] = P[2][7] = 5, no se incluye el elemento 3.
3. i = 2: P[2][7] = 5 ≠ P[1][7] = ∞, se incluye el elemento 2 (peso 3, beneficio 4) y b pasa a 7 − 4 = 3.
4. i = 1: P[1][3] = 2 ≠ P[0][3] = ∞, se incluye el elemento 1 (peso 2, beneficio 3) y b pasa a 0.

Qué muestra el ejemplo

Las dos reconstrucciones devuelven el mismo subconjunto, {1, 2}. El peso mínimo para beneficio 7 es 5, igual a la capacidad W = 5, lo cual verifica la relación entre los planteos mencionada y explicada anteriormente. También se ve la diferencia de tamaño entre las tablas: la del planteo tradicional tiene 6 columnas (W + 1) y la del alternativo tiene 8 (B + 1), porque B = 7 es mayor que W = 5.

3.3 Complejidad

Ambos algoritmos poseen la misma estructura por lo que el análisis es igual para los dos. El algoritmo con el planteo tradicional tiene una complejidad de Θ(n*W) y el alternativo Θ(n*B).

Planteo tradicional:

- Crear e inicializar la tabla de (n+1) * (W+1) celdas cuesta Θ(n*W).
- El doble ciclo for de n iteraciones externas por W internas con comparaciones, operaciones aritméticas y máximo, sucede todo en tiempo constante Θ(n*W).
- Reconstrucción: n pasos de costo Θ(n)
- Total: **Θ(n*W)**

Planteo alternativo:

- Crear e inicializar la tabla de (n+1) * (B+1) celdas, y la columna 0 en cero cuesta Θ(n*B).
- El doble ciclo for de n por B iteraciones de costo constante: Θ(n*B).
- Reconstrucción: Θ(n).
- Total: **Θ(n*B)**

Justificación del conteo. En el planteo tradicional, el ciclo externo recorre los n elementos y, por cada uno, el ciclo interno recorre las W + 1 capacidades. Dentro del ciclo interno solo hay una comparación, a lo sumo una suma y un máximo, y accesos a la tabla por índice. Todas son operaciones de tiempo constante. Por eso la cantidad total de operaciones es proporcional a la cantidad de celdas que se completan:

$T(n, W) = n \cdot (W + 1) \cdot c \in \Theta(n \cdot W)$

donde c es el costo constante de completar una celda. El razonamiento es el mismo para el alternativo, con B en lugar de W: $T(n, B) = n \cdot B \cdot c \in \Theta(n \cdot B)$.

Mejor, peor y caso promedio. Los dos algoritmos completan siempre la tabla entera, sin importar los valores de los pesos y beneficios: no hay cortes anticipados ni podas. Por eso el mejor caso, el peor y el promedio coinciden, y la complejidad es Θ y no solo O.

Complejidad espacial. Ambos algoritmos guardan la tabla completa para poder reconstruir la solución al final. El espacio es Θ(n · W) en el tradicional y Θ(n · B) en el alternativo. La lista de elementos y la lista de elegidos ocupan Θ(n). Si solo se quisiera el valor óptimo, sin los elementos elegidos, alcanzaría con guardar dos filas, y el espacio bajaría a Θ(W) o Θ(B). Pero así no se podría reconstruir la solución.

Explicación de la complejidad **pseudopolinomial** de los algoritmos:

Complejidad pseudopolinomial. Un algoritmo es pseudopolinomial cuando su tiempo es polinomial respecto del valor numérico de los datos de entrada, pero no respecto de su tamaño, es decir, de la cantidad de dígitos o bits necesarios para escribirlos. Es el caso de estos dos algoritmos.

A primera vista, **Θ(n · W)** parece polinomial. Pero n y W no son del mismo tipo: n es una cantidad de elementos, mientras que W es un número que aparece una sola vez en la entrada. Por ejemplo, una capacidad de 1.000 se escribe con 4 dígitos y una de 1.000.000 con 7. Con apenas 3 dígitos más, la tabla pasa a tener 1.000 veces más columnas y el algoritmo tarda 1.000 veces más. Cada dígito que se agrega a W multiplica el tiempo por 10. Medido respecto del tamaño de la entrada, el tiempo crece entonces de forma exponencial. Lo mismo ocurre con B en el planteo alternativo.

Esto significa que los algoritmos son eficientes mientras W o B sean números moderados, como en los sets de este trabajo, y dejan de ser viables cuando esos valores son muy grandes, aunque la cantidad de elementos sea chica. Lo mismo vale para el espacio: como la tabla se guarda completa, la memoria también depende del valor de W o de B, y no solo de n.

3.4 Sets de Datos

Los sets de datos fueron generados con el código provisto por la cátedra crear_mochila.py dentro de un script para que los mismos sean replicables tal como se solicita en la consigna de este TP.

Características de cada set (definidas por crear_mochila.py):

• Capacidad W = 50·n.
• Pesos enteros al azar entre 1 y 200.
• Beneficios enteros al azar entre 1 y 1000.
• Primera línea del archivo: la capacidad. Líneas siguientes: peso,beneficio.

Los sets de datos utilizados fueron elegidos para poder representar el comportamiento de cada algoritmo desde un caso pequeño y más sencillo hasta uno mucho más grande como puede ser el de n = 250, pero sin llegar a un exceso de uso de memoria.

El número de n posible para realizar la evaluación del comportamiento de ambos algoritmos es acotado dado que la tabla se guarda en memoria para luego realizar la reconstrucción. Esto genera que para valores de n muy grandes la capacidad de almacenamiento deba ser cada vez mayor.

Tamaños elegidos: n = 20, 40, 80, 120, 160, 200 y 250, un set por cada tamaño. Para el planteo alternativo, B es el beneficio óptimo del planteo tradicional en ese mismo set

n	W	B	Celdas planteo tradicional	Celdas planteo alternativo
20	1000	7907	21021	166068
40	2000	12778	82041	523939
80	4000	32103	324081	2600424
120	6000	46169	726121	5586570
160	8000	65151	1288161	10489472
200	10000	78095	2010201	15697296
250	12500	97448	3137751	24459699

3.5 Tiempos de ejecución

Metodología de medición. Los tiempos se midieron con el script graficos_pd.py, sobre los siete sets de la sección 3.4. Para cada set se ejecutó primero el planteo tradicional con la capacidad W del archivo, y su beneficio óptimo se usó como beneficio objetivo B del planteo alternativo. Todas las mediciones se hicieron en la misma computadora (MacBook Air con procesador Apple Silicon, macOS, Python 3.14.2), y los resultados quedan guardados en resultados/resultados_pd.csv.

Resultados de las mediciones:

n	W	B	t tradicional (s)	t alternativo (s)	peso mínimo
20	1000	7907	0,0018	0,0141	993
40	2000	12778	0,0071	0,0454	1986
80	4000	32103	0,0310	0,2292	3998
120	6000	46169	0,0727	0,4971	5994
160	8000	65151	0,1315	0,9412	7991
200	10000	78095	0,2308	1,4458	9997
250	12500	97448	0,3661	2,2402	12497

Tabla. Tiempos de ejecución de ambos planteos, en segundos. El peso mínimo del alternativo es en todos los casos menor o igual que W, lo que confirma que ambos algoritmos llegan al mismo beneficio óptimo.

Gráfico del algoritmo tradicional.

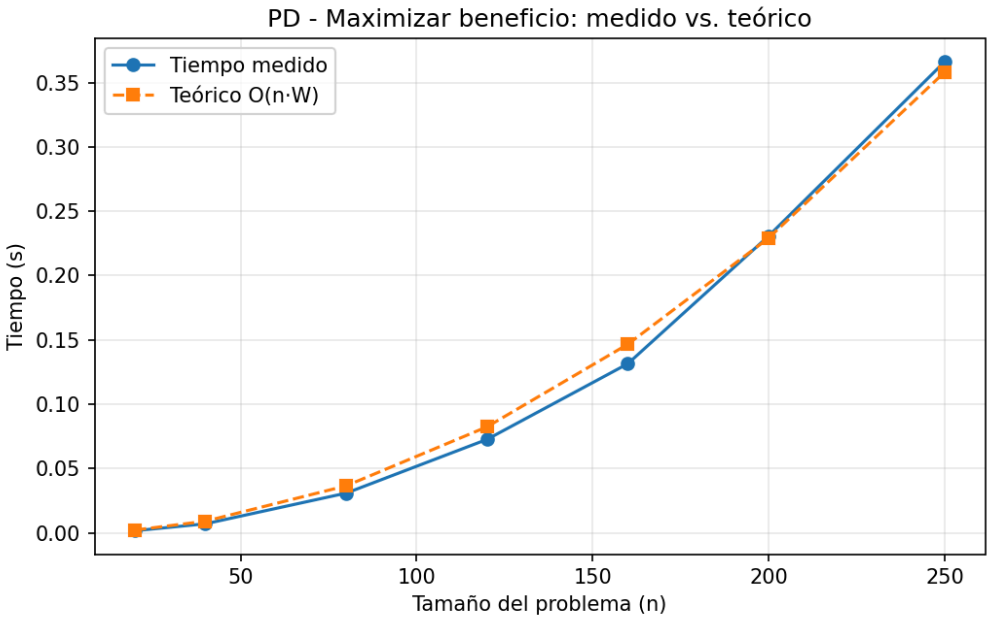


Gráfico del planteo tradicional. El eje horizontal es la cantidad de elementos n, y el vertical, el tiempo en segundos. La línea azul son los tiempos medidos, y la naranja punteada, la curva teórica proporcional a n · W. Las dos curvas se superponen en todo el rango y tienen forma cuadrática, porque W = 50 · n y entonces n · W = 50 · n².

Gráfico del algoritmo Alternativo

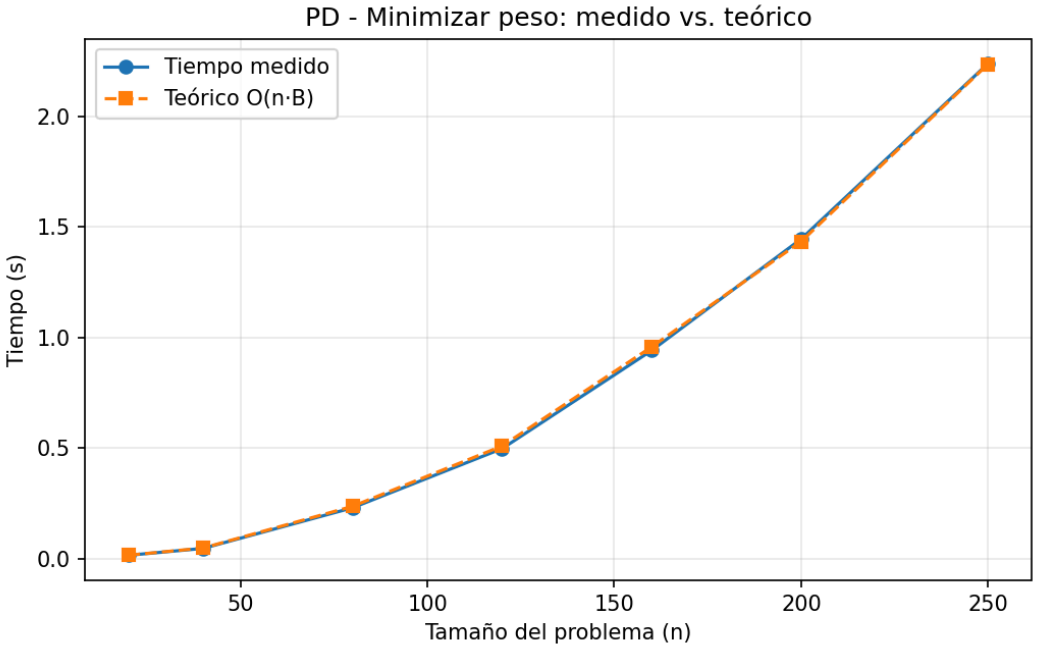


Gráfico del planteo alternativo. Mismos ejes, con la curva teórica proporcional a n · B. La coincidencia es todavía mayor que en el tradicional: los puntos medidos quedan prácticamente sobre la curva teórica en todos los sets. Los tiempos son entre 6 y 8 veces mayores que en el tradicional, porque la tabla tiene muchas más celdas.

Gráfico comparativo de los dos planteos:

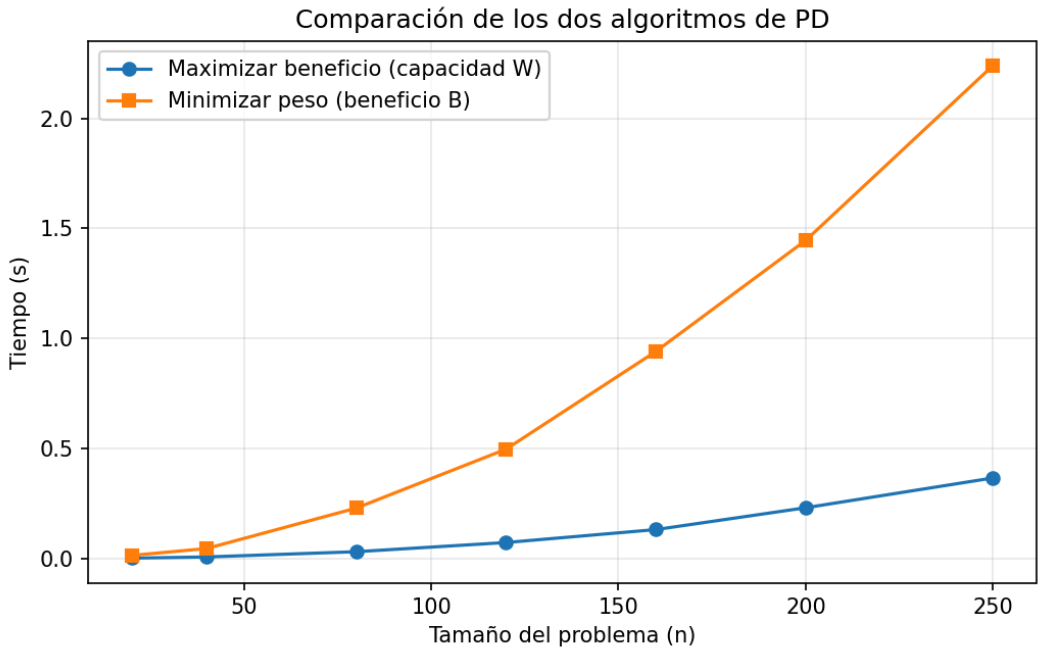


Gráfico comparativo. Muestra los tiempos medidos de ambos planteos en la misma escala. Las dos curvas crecen de forma cuadrática, pero la del alternativo se separa cada vez más. Para n = 250, el tradicional tarda 0,37 segundos y el alternativo 2,24.

3.6 Informe de resultados

A partir de las mediciones realizadas con el set de datos elegido podemos observar que para ambos planteos, tanto el tradicional como el alternativo, las curvas dadas a partir de los valores utilizados para la medición se corresponden claramente con la curva teórica de su complejidad.

Las mayores diferencias las podemos encontrar en la comparación entre los dos planteos. Si bien tienen la misma forma de complejidad, Θ(n*W) para el tradicional y Θ(n*B) para el alternativo, el algoritmo del planteo alternativo posee un tiempo de ejecución mayor de aproximadamente B/W veces el tradicional. Esto se condice con lo esperado: La cátedra proporcionó, en el algoritmo de generación de la mochila, valores de beneficio entre 1 y

1000 y de peso entre 1 y 200, con una capacidad W = n * 50. Como el beneficio objetivo B se tomó como el beneficio óptimo del planteo tradicional, este resulta mucho mayor que W. Por lo tanto, aunque el costo por celda para cada algoritmo sea el mismo, el ancho de la tabla para el planteo alternativo será mucho más amplio respecto al tradicional y tiene más celdas para completar.

Relación entre los dos planteos. Como el trabajo por celda es parecido en ambos algoritmos, el tiempo depende principalmente de la cantidad de celdas de la tabla. Por eso, cuántas veces más tarda el alternativo (t_2 / t_1) debería parecerse a cuántas veces más ancha es su tabla (B / W).

n	B / W	t_2 / t_1
20	7,91	7,83
40	6,39	6,39
80	8,03	7,39
120	7,69	6,84
160	8,14	7,16
200	7,81	6,26
250	7,80	6,12

Por ejemplo, para n = 250: W = 12500 y B = 97448, por lo que B / W ≈ 7,80. El tradicional tardó t_1 = 0,3661 s y el alternativo t_2 = 2,2402 s, un cociente de 6,12. Las dos columnas siguen la misma tendencia: el set de n = 40, que tiene la tabla relativamente más angosta (B / W = 6,39), es también el de menor cociente de tiempos entre los primeros tamaños. Esto confirma que la diferencia de tiempo se explica por el ancho de la tabla.

Limitaciones. Ambos algoritmos guardan la tabla completa para poder reconstruir la solución, así que la memoria crece igual que el tiempo. En n = 250, la tabla del alternativo ya tiene más de 24 millones de celdas. Los tiempos absolutos dependen de la computadora y pueden variar levemente entre ejecuciones, pero la forma de las curvas y la relación entre ambos planteos se mantienen.

3.7 Conclusiones

Los dos algoritmos de programación dinámica resuelven el problema de la mochila 0/1 de forma exacta, y sus tiempos de ejecución se corresponden con la complejidad calculada: $\Theta(n \cdot W)$ para el planteo tradicional y $\Theta(n \cdot B)$ para el alternativo. En los dos gráficos, la curva medida y la teórica prácticamente se superponen.

Aunque ambos tienen la misma estructura, el planteo tradicional resultó entre 6 y 8 veces más rápido en estos sets. La diferencia no viene del algoritmo sino de la escala de los datos: con los rangos de crear_mochila.py, el beneficio óptimo B es unas 8 veces mayor que la capacidad W, y por eso la tabla del alternativo es mucho más ancha. El alternativo convendría en el caso contrario, con capacidades muy grandes y beneficios chicos. En general, conviene el planteo cuyo valor numérico (W o B) sea menor.

Ambos algoritmos son pseudopolinomiales: su costo depende de la magnitud de W o de B, y no solo de la cantidad de elementos. Esto les da un tiempo predecible, que no depende de cómo estén distribuidos los datos, y los hace mucho más eficientes que la fuerza bruta, que necesita $\Theta(n \cdot 2^n)$. A cambio, el tiempo y sobre todo la memoria limitan su uso cuando W o B son muy grandes.

4. Programación lineal

4.1 Supuestos

Para resolver el problema mediante programación lineal se tienen en cuenta los siguientes supuestos:

- Los elementos de la mochila o están o no están, no se permiten fracciones
- Tanto los valores peso w_i y valor v_i de los elementos tienen que ser positivos, además del tamaño total W de la mochila
- La resolución del problema se realizará utilizando la biblioteca Pulp

4.2 a. Ecuación fundamental y pseudocódigo

Para poder plantear el problema mediante programación lineal y resolverlo primero tenemos que armar la función objetivo y las restricciones, en el caso del problema de la mochila se verían de la siguiente manera:

-Variable: la única variable que vamos a utilizar es para saber si un ítem está o no en la mochila, esto lo podemos representar como una variable binaria x_i (1 si llevo el elemento, 0 si no lo llevo):

$x \in \{0, 1\}$

-Ecuación fundamental: partiendo de la variable ahora podemos definir la ecuación fundamental, que en este caso es de maximización ya que lo que queremos obtener es el mayor beneficio posible entre los elementos capaces de entrar en nuestra mochila sin superar la capacidad de la misma:

$$Z(max) = \sum_{i=1}^{n} v_i x_i$$

-Restricciones: en este caso la restricción va a estar delimitada por la suma total del peso de todos los elementos dentro de la mochila y hacer que estos no sobrepasen el peso máximo:

$$\sum_{i=1}^{n} w_i x_i \leq W$$

Con las ecuaciones ya planteadas podemos escribir el pseudocódigo:

```
mochilaProgramacionLineal(items, capacidadMochila):
    n = longitud(items)
    problema = CrearProblema(maximización, “Mochila”)
    x[] (variables)
    beneficio[] (beneficio de los ítems)
    peso[] (peso de los ítems)

    for i in range n:
        variable = crearVariable(Item_n, variableBinaria)
        agregar variable a x

        beneficio_item = item[i] beneficio
        agregar (beneficio_item * variable) a beneficio

        peso_item = item[i] peso
        agregar (peso_item * variable) a peso
    end for

    problema.objetivo(suma(beneficios))
    problema.restricción(suma(pesos) <= capacidadMochila)

    problema.resolver
    return resultado
```

b. Estructuras de datos utilizadas

- Listas: las usamos para tener la lista total de ítems posibles
- Tuplas: Las usamos para representar los atributos de cada ítem (peso, beneficio)

4.3 Seguimiento

Para ver cómo funciona, vamos a resolver un ejemplo de 4 objetos con una mochila que soporta hasta 10 de peso (W = 10). Antes recordemos que pulp resuelve los ejercicios con branch and bound.

Primero ordenamos los objetos desde el más rendidor hasta el menos rendidor, dividiendo su valor por su peso:

- Ítem 1: pesa 4, vale 40
- Ítem 2: pesa 7, vale 42
- Ítem 3: pesa 5, vale 25
- Ítem 4: pesa 3, vale 12

Como no podemos partir objetos, el algoritmo usa dos valores de referencia en cada paso para descartar combinaciones malas rápidamente:

-Piso(cota inferior): Es la ganancia de la mejor combinación válida (con objetos enteros) que ya hayamos encontrado hasta el momento.

-Techo(cota superior): Es el cálculo de cuánto ganaríamos en el mejor de los casos si pudiéramos fraccionar objetos para llenar la mochila al 100%.

Paso 1: Punto de partida (raíz), calculamos el techo permitiendo fraccionar los objetos y llenando la mochila:

- Ítem 1: entra completo. Ocupa 4 de peso y sobran 6 de capacidad.
- Ítem 2: no entra completo porque pesa 7 y solo queda 6 de espacio. Entra la fracción que ocupa: 6/7. Capacidad restante = 0.

Beneficio de la raíz = 40 * (1) + 42 * (6/7) + 25 * (0) + 12 * (0) = 40 + 36 = 76. Por lo tanto, el techo = 76, es decir, es imposible superar 76 sin partir objetos.

Ahora buscamos la solución para el piso. Armamos una mochila válida usando solo objetos enteros, si llevamos el Ítem 1 y el Ítem 3 $\{x_1 = 1, x_3 = 1\}$, el peso suma 4 + 5 = 9 <= 10, y obtenemos un beneficio de 40 + 25 = 65. Se establece el Piso = 65 (cualquier camino futuro que dé 65 o menos se va a descartar).

Paso 2: Ramificación de la variable x_2.

En el Paso 1, el ítem 2 quedó partido (x_2 = 6/7). Como en la realidad no se puede partir, dividimos el problema en dos ramas:

- Rama Derecha: lleva el ítem 2 entero ($x_2 = 1$).
- Rama Izquierda: no lleva el ítem 2 ($x_2 = 0$).

Paso 3: Exploración de la rama derecha $x_2 = 1$:

Empezamos con la mochila vacía y metemos el ítem 2 (pesa 7 y suma 42 de beneficio). La capacidad libre que nos queda para rellenar con los demás ítems es 3. Calculamos el techo de esta rama con los ítems restantes:

- El más rendidor disponible es el ítem 1 (pesa 4). Como solo quedan 3 de espacio libre, entra fraccionado $x_1 = 3/4$.
- Techo = 42 * (1) + 40 * (3/4) = 42 + 30 = 72

Como este techo supera nuestro piso actual (65), vale la pena revisar qué pasa si no partimos el ítem 1. Dividimos en dos subramas sobre x_1:

a) Subrama x_1 = 1 (junto con x_2 = 1): Sumamos los pesos de llevar ambos enteros: 7 + 4 = 11 > 10. → Poda por exceso de peso (supera la capacidad de la mochila).
b) Subrama x_1 = 0 (junto con x_2 = 1): Seguimos con 3 de espacio libre y probamos rellenar con los candidatos restantes {3, 4}. El Ítem 3 (peso 5) no entra entero en los 3 libres, por lo que entra fraccionado con x_3 = 3/5. Techo = 42 + 25 * (3/5) = 57. Como este techo ni siquiera alcanza el piso de 65 que ya tenemos, ninguna combinación entera acá adentro podrá mejorar lo que ya tenemos. → Poda por cota (baja ganancia).

Paso 4: Exploración de la Rama Izquierda ($x_2 = 0$)

Empezamos con la mochila vacía (capacidad completa W = 10). Rellenamos en orden con los ítems candidatos {1, 3, 4}: item 1 entra con x_1 = 1 (pesa 4, sobran 6 de espacio), el ítem 3 entra completo con x_3 = 1 (pesa 5, sobra 1 de espacio) y el ítem 4 pesa 3 pero solo queda

1 de espacio libre, por lo que entra fraccionado con x_4 = 1/3. Techo = 40 * (1) + 25 * (1) + 12 * (1/3) = 65 + 4 = 69.

Como el techo (69) es mayor a nuestro piso actual (65), abrimos dos subramas para eliminar la fracción del Ítem 4:

a) Si x_4 = 0: Quedan dentro solo el ítem 1 y el ítem 3 enteros, dando la asignación x = (1, 0, 1, 0). Peso total = 4 + 5 = 9 <= 10. Beneficio = 40 + 25 = 65. Como todos los valores ya son enteros y válidos → Poda porque ya es una solución.
b) Si x_4 = 1: El ítem 4 ocupa 3 de peso (beneficio 12) y deja 7 libres. El Ítem 1 entra completo (pesa 4, suma 40 y deja 3 libres). El Ítem 3 entra fraccionado en esos 3 libres (x_3 = 3/5, suma 15), dando un techo = 12 + 40 + 15 = 67. Al obligar a que el Ítem 3 sea entero: si x_3 = 0 da un beneficio de 12 + 40 = 52 < 65 (se descarta por cota), si x3 = 1: el peso total suma 3 + 4 + 5 = 12 > 10 (se descarta por exceso de peso).

Paso 5: Conclusión y Solución Óptima

Una vez ya se analizaron todas las ramas posibles y no quedan nodos activos con un techo mayor = 65. Obtenemos la mejor solución posible que es:

- Asignación óptima: x = (1, 0, 1, 0), es decir, ítems seleccionados: 1 y 3
- Peso total utilizado: 4 + 5 = 9 (<= 10, sobra 1 unidad de capacidad libre)
- Beneficio óptimo: Z = 65

4.4 Complejidad

La complejidad del algoritmo depende principalmente de la instrucción problema.solve() que, como mencionamos en la sección anterior, está determinado por branch and bound

Como cada elemento tiene solo dos opciones (se guarda en la mochila o no), por cada elemento nuevo el árbol se divide en dos, duplicando los caminos. En el peor escenario posible, el algoritmo no logra descartar ninguna opción y se ve obligado a recorrer hasta el final cada una de las ramas del árbol, evaluando 2^{n+1} nodos y revisando todas las

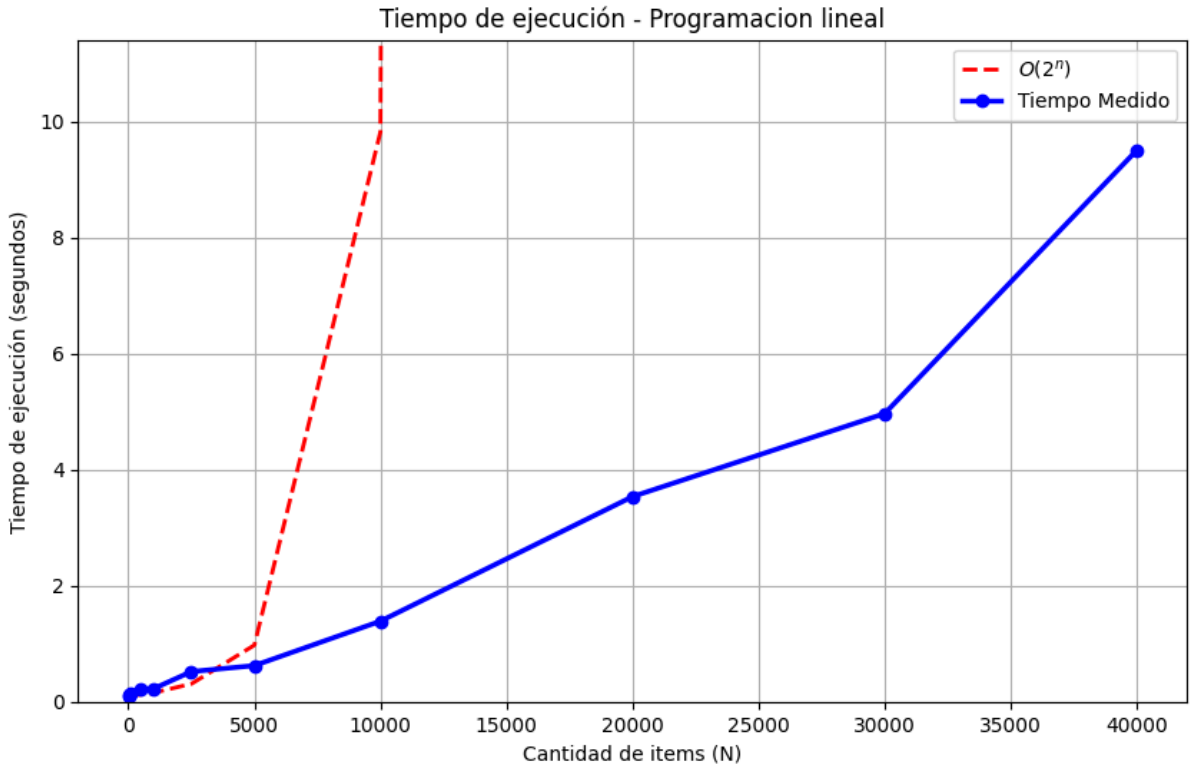
combinaciones posibles. Es por todo esto que la complejidad teórica en el peor de los casos es $O(2^n)$.

Sin embargo, en la práctica es extremadamente raro que tarde tanto. El solver utiliza estimaciones matemáticas que le permiten saber anticipadamente qué ramas del árbol no tienen una buena solución. Gracias a esto logra "podar" esos caminos muertos desde los primeros niveles, permitiendo encontrar la respuesta correcta evaluando menos nodos en lugar de tener que explorar el árbol completo.

4.5 Set de datos

Para este caso se eligió un set de datos que incluye los siguientes valores (cada valor significa la cantidad de ítems generados aleatoriamente), el tamaño de la mochila es n * 50, pesos entre 1 y 200 y valores entre 1 y 1000:
[10, 50, 100, 500, 1000, 2500, 5000, 10000, 20000, 30000, 40000]

4.6 Tiempos de ejecución:



Los tiempos obtenidos fueron ≃

[0.09, 0.12, 0.14, 0.22, 0.22, 0.51, 0.62, 1.38, 3.53, 4.96, 9.50]

4.7 Informe de los resultados

En el gráfico comparamos los tiempos medidos (en azul) contra la curva teórica $O(2^n)$. Elegimos este gráfico como referencia porque representa el peor escenario teórico posible del algoritmo, es decir, el caso en el que branch and bound no lograría podar ningún camino y se vería forzado a recorrer todas las combinaciones del árbol completo. De esta forma se puede contrastar el peor caso teórico contra lo que realmente tarda el programa en ejecutarse.

Para valores chicos de n la curva teórica parece acompañar a las mediciones porque los tiempos son muy chicos. Sin embargo, mientras n crece la curva roja explota verticalmente y se escapa del gráfico. Esto muestra claramente por qué un crecimiento exponencial puro es inviable.

Por el contrario, la curva azul con los tiempos medidos continúa creciendo de manera suave y casi recta hasta llegar a n=40.000, tardando apenas 9,5 segundos. Esto demuestra que en la práctica el algoritmo no recorre el árbol completo gracias a que descarta las ramas inviables, evaluando pocos nodos y permitiendo resolver problemas grandes en pocos segundos. En conclusión, el gráfico evidencia que la cota $O(2^n)$ es solo un límite teórico extremo que no refleja el comportamiento promedio.

Conclusiones

Para comparar los seis algoritmos sobre los mismos problemas se usaron los sets de crear_mochila.py del Ejercicio 1 (n entre 10 y 2000, generados con semilla fija). Cada implementación se ejecutó sin modificaciones mediante el script Comparacion/comparar.py, en la misma computadora (MacBook Air con procesador Apple Silicon, macOS, Python 3.9.6, PuLP 3.3.1 y el solver CBC 2.10.13). Cada algoritmo se corrió hasta el tamaño en que su tiempo o su memoria seguían siendo razonables: la fuerza bruta hasta n = 22, la programación dinámica alternativa hasta n = 200, la tradicional hasta n = 500 y el resto hasta n = 2000. Como en el Ejercicio 1, las ejecuciones de menos de 0,5 segundos se repitieron cinco veces y se registró el tiempo mínimo. En el greedy, el tiempo incluye el ordenamiento por beneficio/peso, y en la programación dinámica alternativa se usó como beneficio objetivo B el óptimo de cada instancia. Los resultados completos están en Comparacion/resultados/comparacion.csv.

Tabla 1. Tiempos de ejecución, en segundos, de los seis algoritmos sobre los mismos sets.

n	Fuerza bruta	Back-tracking	Greedy	PD tradicional	PD alternativa	Prog. lineal
10	0,00075	0,000017	0,0000022	0,00084	0,0062	0,0092
22	6,32	0,000054	0,0000048	0,0045	0,037	0,0093
100	—	0,00040	0,000018	0,097	0,78	0,012
200	—	0,0017	0,000036	0,41	3,00	0,025
500	—	0,0080	0,000089	2,77	—	0,049
1000	—	0,027	0,00018	—	—	0,056
2000	—	0,096	0,00035	—	—	0,115

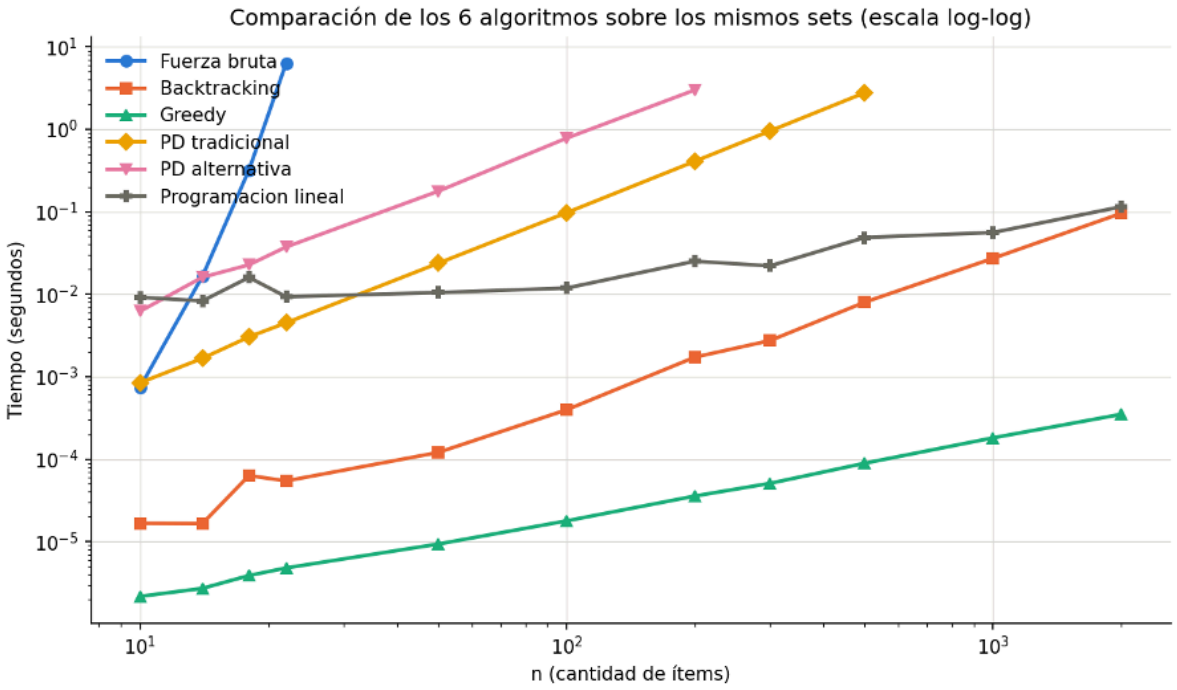


Figura. Tiempos de los seis algoritmos sobre los mismos sets, en escala log-log.

En cuanto a los resultados obtenidos, todos los algoritmos exactos coincidieron en todos los sets: la fuerza bruta, el backtracking, la programación dinámica tradicional y la programación lineal encontraron el mismo beneficio óptimo, y la programación dinámica alternativa encontró en cada caso un subconjunto con ese beneficio. El único que no siempre llega al óptimo es el greedy, que en n = 22 obtuvo el 98,3% del beneficio óptimo y en n = 100 el 99,8%; a partir de n = 200 superó siempre el 99,9%. Esto es consistente con su garantía de 1/2, que es una cota para el peor caso: en estas instancias, la parte de la capacidad que el greedy deja sin aprovechar es muy chica en comparación con el beneficio total.

En cuanto a los tiempos, el orden entre los algoritmos coincide con sus complejidades. La fuerza bruta, $\Theta(n \cdot 2^n)$, es la única cuya curva crece casi verticalmente en el gráfico: pasa de 0,00075 segundos en n = 10 a 6,32 segundos en n = 22, y por eso no se pudo medir más allá. Las dos programaciones dinámicas aparecen como rectas de pendiente 2 en escala log-log, es decir, su tiempo crece como n^2. Esto se explica porque sus complejidades son $\Theta(n \cdot W)$ y $\Theta(n \cdot B)$, y en los sets de crear_mochila.py tanto la capacidad $W = 50 \cdot n$ como el beneficio óptimo B crecen proporcionalmente a n. La alternativa resultó entre 7 y 8 veces más lenta que la tradicional en todo el rango, la misma relación que hay entre B y W, como se analizó en la sección 3.6. Ninguna de las dos llega a n = 2000: la tradicional ya tarda

2,77 segundos en n = 500 y la alternativa 3 segundos en n = 200, además de que ambas guardan la tabla completa en memoria.

El backtracking y la programación lineal tienen complejidad exponencial en el peor caso, pero en estos sets se comportan como algoritmos polinomiales: los dos resuelven n = 2000 en alrededor de una décima de segundo, porque ambos descartan ramas del árbol de soluciones usando la cota de la relajación fraccional. El backtracking fue el más rápido de los dos en todo el rango, aunque la diferencia se achica a medida que crece n: en n = 10 la programación lineal es 550 veces más lenta, pero en n = 2000 solo 1,2 veces. Esto se debe a que la programación lineal tiene un costo fijo cercano a 0,01 segundos, que corresponde a armar el modelo y ejecutar el solver externo, y que domina en las instancias chicas.

El greedy fue el más rápido en todos los tamaños, entre 8 y 270 veces más rápido que el backtracking, y su tiempo crece casi linealmente con n, como corresponde a su complejidad O(n log n).

En conclusión, si se necesita la solución exacta, en instancias como las de crear_mochila.py el backtracking y la programación lineal son las mejores opciones: el backtracking es más rápido en todo el rango medido, y la programación lineal resulta más simple de plantear, aunque depende de un solver externo. Si alcanza con una solución muy cercana al óptimo, el greedy es la opción más rápida. La programación dinámica tiene la ventaja de que su tiempo es predecible y no depende de cómo son los datos, pero sí de la magnitud de W o de B, lo que la limita en tiempo y en memoria. La fuerza bruta solo es viable para instancias muy chicas y sirve como referencia para verificar los resultados de los demás algoritmos. En definitiva, ninguna técnica es la mejor en todos los casos: la elección depende de si se necesita el óptimo exacto, del tamaño de la instancia y de la estructura de los datos.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Curso Echevarría, Teoría de Algoritmos. (2026a). *Backtracking: Parte I* [Diapositivas de clase]. Facultad de Ingeniería, Universidad de Buenos Aires.

Curso Echevarría, Teoría de Algoritmos. (2026b). *Backtracking: Parte II* [Diapositivas de clase]. Facultad de Ingeniería, Universidad de Buenos Aires.